

# PieClam: Supplementary Material (After the main paper)

Anonymous Authors

<sup>1</sup>Association for the Advancement of Artificial Intelligence  
1101 Pennsylvania Ave, NW Suite 300  
Washington, DC 20004 USA  
proceedings-questions@aaai.org

## Abstract

We propose PieClam (Prior Inclusive Exclusive Cluster Affiliation Model): a probabilistic graph model for representing any graph as overlapping generalized communities. Our method can be interpreted as a graph autoencoder: nodes are embedded into a code space by an algorithm that maximizes the log-likelihood of the decoded graph, given the input graph. PieClam is a community affiliation model that extends well-known methods like BigClam in two main manners. First, instead of the decoder being defined via pairwise interactions between the nodes in the code space, we also incorporate a learned prior on the distribution of nodes in the code space, turning our method into a graph generative model. Secondly, we generalize the notion of communities by allowing not only sets of nodes with strong connectivity, which we call inclusive communities, but also sets of nodes with strong disconnection, which we call exclusive communities. To model both types of communities, we propose a new type of decoder based the Lorentz inner product, which we prove to be much more expressive than standard decoders based on standard inner products or norm distances. By introducing a new graph similarity measure, that we call the log cut distance, we show that PieClam is a universal autoencoder, able to uniformly approximately reconstruct any graph. Our method is shown to obtain competitive performance in graph anomaly detection benchmarks.

## 1 Introduction

In recent years, considerable research has concentrated on graph representation learning, aiming to develop vector representations for graph entities, including nodes, edges, and subgraphs (??). In graph autoencoders, e.g., (????), the vertices of a graph are embedded in a code space, where edges are inferred from the locations of the vertices in this space. Encoding graphs into a standard space has a number of advantages. While different graphs can have different sizes and topology, the code space is fixed with a fixed dimension. This helps when learning downstream tasks, where the representation of the graph in the code space can be processed. For example, in link prediction, one infers unknown edges by defining (?) or learning (?) a function that takes pairs of nodes in the code space and predicts if there is an edge between them. In anomaly detection, one defines (??) or learns

(??), a function that takes the representation of a node and its neighborhood and predicts if this node is normal or an anomaly. In graph and node classification, the graph is represented in the code space, and one learns a model that predicts from this representation the classes of the nodes or of the graph (??).

**Our Contribution.** In this paper, we derive a new graph autoencoder from a statistical model of graphs. In our model, similarly to SBM (??) or community-based statistical models (??), graphs are generated from a combination of intersecting communities/cliques. Namely, each node belongs to a different subset of a predefined set of communities, and the affiliations of the nodes to the different communities determine the probabilities of the edges of the graph. Here, the estimation of the community affiliations is seen as an encoder of graphs to a *community affiliation space*, and the computation of the corresponding edge probabilities is seen as a decoder. As opposed to past works, we consider two types of *generalized communities*. First, standard *inclusive communities*, where any two nodes in the same community are likely to be connected. Second, we propose *exclusive communities*, where belonging to the same community reduces the probability of nodes being connected. For illustration, consider a social network of employers/recruiters and employees/job-seekers. Such a network has roughly bipartite components, where job-seekers do not tend to connect to other seekers, and employers do not connect to other employers, but employers and seekers of the same subsector tend to connect. Hence, inclusive communities for such a graph can correspond to job titles or subsectors, and exclusive communities can correspond to sets of job-seekers and sets of employees from the same sector.

To formalize the above ideas, we propose the *Prior Inclusive Exclusive Cluster Affiliation Model (PieClam)*. This model represents graphs as overlapping inclusive and exclusive communities. Moreover, instead of embedding the nodes of a given graph in the community code space, our model also learns a prior on the code space, so new graphs can be generated from the learned model. This makes PieClam a graph generative model, like, e.g., (????).

To model both types of communities, we propose a new type of decoder based on the *Lorentz inner product*, in which case the code space is typically called a *pseudo Euclidean*

space (?). The addition of exclusive communities in our model is not merely aimed at improving it heuristically for special graphs like social networks. Rather, we prove in Theorems ?? and ?? that using exclusive communities (via the Lorentz inner product) makes our model universal, namely, able to approximate with a fixed budget of parameters any graph. This is in contrast to standard decoders based on only inclusive communities, which we show are unable to represent many graphs.

To formalize this universality property, we propose a new similarity measure between graphs with edge probabilities, that we call the *log cut distance*. We formalize the universality of our model as follows: one can choose the dimension of the code space (the number of communities) a priori, and guarantee that any graph of any size and topology can be approximated up to small log cut distance by decoding points from this fixed space. We show that other related decoders do not satisfy this universality property.

In Section ?? we support our construction with experiments. We first conduct some toy experiments that illustrate the merits of our method. We then use PieClam to perform graph anomaly detection. Here, PieClam learns a probabilistic model corresponding to a given graph, and this model can be directly used for inspecting the probabilities of different nodes in this graph. Nodes with low probabilities are deemed to be anomalies. Our models achieve competitive performance with respect to state of the art anomaly detection methods.

Appendix ?? has an extended discussion on related work.

## 2 Community Affiliation Models With Prior

Our model PieClam is best understood as an extension of the BigClam model (?). Hence, after introducing basic notations, we start with a detailed exposition of BigClam, before introducing our novel constructions.

### 2.1 Notations

We denote by  $\mathbb{R}$  the real line, and by  $\mathbb{R}_+$  the non-negative real numbers. We denote “and” by  $\wedge$ . We denote matrices as boldface capital letter  $\mathbf{B} = \{b_{n,m}\}_{n,m}$ , vectors as boldface lowercase letters  $\mathbf{b} = \{b_n\}_n$ , and scalars as regular lowercase letters  $b$ . Vectors  $\mathbf{b} \in \mathbb{R}^N$  are always column vectors, and their transpose  $\mathbf{b}^\top$  row vectors. The rows of a matrix  $\mathbf{B} \in \mathbb{R}^{N \times C}$  are denoted by the same letter in lowercase  $\mathbf{b}_n^\top$ , where  $\mathbf{b}_n \in \mathbb{R}^C$ , where we write in short  $\mathbf{b}_n^\top \in \mathbb{R}^C$ . A diagonal matrix  $\mathbf{D} \in \mathbb{R}^{N \times N}$  with diagonal entries  $\mathbf{d}$  is denoted by  $\text{diag}(\mathbf{d}) = \text{diag}(d_1, \dots, d_N)$ . The  $\ell^2$  norm of a vector  $\mathbf{b} \in \mathbb{R}^N$  is defined to be  $\|\mathbf{b}\| = (\sum_{n=1}^N b_n^2)^{1/2}$ .

A graph is denoted by  $G = ([N], E, \mathbf{A})$ , where  $[N] = \{1, \dots, N\}$  is the set of  $N$  nodes,  $E \subseteq [N] \times [N]$  is the set of edges, and  $\mathbf{A} \in \{0, 1\}^{N \times N}$  is the adjacency matrix. For weighted graphs,  $\mathbf{A} \in [0, 1]^{N \times N}$ , where  $a_{n,m}$  is the edge weight of  $(n, m)$ . In this work we focus on undirected graphs, for which  $(m, n) \in E \Leftrightarrow (n, m) \in E$ , and  $\mathbf{A} = \mathbf{A}^\top$ . Any pair  $(n, m) \in [N] \times [N]$  is called a *dyad*. The neighborhood of a node  $n \in [N]$  is  $\mathcal{N}(n) = \{m \in [N] \mid (m, n) \in E\}$ . A graph-signal is a graph with node features  $G = ([N], E, \mathbf{A}, \mathbf{X})$  where  $\mathbf{X} = \{\mathbf{x}_n^\top\}_{n=1}^N \in \mathbb{R}^{N \times D}$ ,  $\mathbf{x}_n \in$

$\mathbb{R}^D$ , and  $D$  is called the feature dimension. A random graph is a graph-valued random variable. Given a random graph with nodes  $[N]$ , we denote the event in which  $(n, m) \in E$  by  $n \sim m$ , and the event  $(n, m) \notin E$  by  $\neg(n \sim m)$ . A graph is bipartite if its vertex set  $[N]$  can be partitioned into two disjoint sets  $\mathcal{U}$  and  $\mathcal{V}$ , with  $\mathcal{U} \cup \mathcal{V} = [N]$ , such that every edge has one endpoint in  $\mathcal{U}$  and the other in  $\mathcal{V}$ .

### 2.2 BigClam

The code space in BigClam is  $\mathbb{R}_+^C$ , where each axis is interpreted as a community. Each entry  $f^c$  of a point  $\mathbf{f} = (f^1, \dots, f^C) \in \mathbb{R}_+^C$  is interpreted as how much the point belongs to community  $c$ , where 0 means “not included in  $c$ ” and 1 “included.” In this paper we call  $\mathbb{R}_+^C$  the *affiliation space (AS)*, and call any point in the affiliation space an *affiliation feature (AF)*.

BigClam is a model where a simple random graph is decoded from a sequence of AFs  $\mathbf{F} = \{\mathbf{f}_n = (f_n^1, \dots, f_n^C)\}_{n=1}^N$  in the AS. For each pair of nodes  $n, m \in [N]$ , the probability of the event  $\neg(n \sim m)$  (no edge between  $n$  and  $m$ ), given their amount of membership in the same community  $c$ , is defined to be

$$P(\neg(n \sim m) | f_n^c, f_m^c) = e^{-f_n^c f_m^c}.$$

BigClam makes two assumptions of independence. First, the membership in one community does not affect the membership in another community. Hence,

$$P(\neg(n \sim m) | \mathbf{F}) = P(\neg(n \sim m) | \mathbf{f}_n, \mathbf{f}_m) = e^{-\mathbf{f}_n^\top \mathbf{f}_m}, \quad (1)$$

$$P(n \sim m | \mathbf{F}) = P(n \sim m | \mathbf{f}_n, \mathbf{f}_m) = 1 - e^{-\mathbf{f}_n^\top \mathbf{f}_m}.$$

Due to this formula, BigClam is a so called *Bernoulli-Poisson model* (see Appendix ?? for more details). The second assumption is that the events of having an edge between different pairs of nodes are independent. As a result, the probability of the entire graph  $([N], E)$  conditioned on the AFs  $\mathbf{F}$  is

$$P(E | \mathbf{F}) = \prod_{n \in [N]} \left( \sqrt{\prod_{m \in \mathcal{N}(n)} P(n \sim m | \mathbf{F}) \prod_{m \notin \mathcal{N}(n)} P(\neg(n \sim m) | \mathbf{F})} \right) \quad (2)$$

Here, the square root is taken because the product considers each edge twice.

BigClam consist of a decoder, decoding from the AFs  $\mathbf{F}$  the random graph  $G(\mathbf{F})$  with node set  $[N]$  and independent Bernoulli distributed edges with probabilities  $\mathbf{P} = \{P(n \sim m | \mathbf{f}_n, \mathbf{f}_m)\}_{n,m=1}^N$ .

From the encoding side, given a simple graph  $([N], E)$ , BigClam encodes the graph into the affiliation space by maximizing the log likelihood with respect to the AFs  $\mathbf{F}$

$$l(\mathbf{F}) = \frac{1}{2} \sum_{n \in [N]} \left( \sum_{m \in \mathcal{N}(n)} \log(1 - e^{-\mathbf{f}_n^\top \mathbf{f}_m}) - \sum_{m \notin \mathcal{N}(n)} \mathbf{f}_n^\top \mathbf{f}_m \right). \quad (3)$$

BigClam is optimized by gradient descent, with update at iteration  $i$

$$\mathbf{F}^{(i+1)} = \mathbf{F}^{(i)} + \delta \nabla_{\mathbf{F}} l(\mathbf{F}^{(i)}), \quad (4)$$

for some learning rate  $\delta > 0$ . In order to implement the above iteration with  $O(|E|)$  operations at each step, instead of  $O(N^2)$ , the loss can be rearranged as

$$2l(\mathbf{F}) = \sum_{n \in [N]} \left( \sum_{m \in \mathcal{N}(n)} \log(e^{\mathbf{f}_n^\top \mathbf{f}_m} - 1) - \mathbf{f}_n^\top \sum_{m \in [N]} \mathbf{f}_m + \|\mathbf{f}_n\|^2 \right). \quad (5)$$

The gradient of the loss is now

$$\nabla_{\mathbf{f}_n} l = \sum_{m \in \mathcal{N}(n)} \mathbf{f}_m (1 - e^{-\mathbf{f}_n^\top \mathbf{f}_m})^{-1} - \sum_{m \in [N]} \mathbf{f}_m + \mathbf{f}_n. \quad (6)$$

Since the global term needs to be calculated only once, the number of operations is  $O(|E|)$  instead of  $O(N^2)$ .

We observe that the optimization process is a message passing scheme. Looking at the dynamics of the optimization process, we see that every node is pushed in the direction of a weighted average of all of its neighbors, and pushed in the opposite direction by an average of all of the nodes. The sum of both forces tends to drive communities towards the axes in the optimization dynamics.

### 2.3 Inclusive-Exclusive Cluster Affiliation Model

The BigClam decoder has a limitation due to a ‘‘triangle inequality type’’ behavior. Namely, suppose that we would like to construct two features  $\mathbf{f}_1$  and  $\mathbf{f}_2$  with strong connectivity to a third feature  $\mathbf{f}_3$ , and we are limited by a fixed affiliation feature dimension  $C$ . A naive approach to achieve this would be to put  $\mathbf{f}_1$  and  $\mathbf{f}_2$  close to  $\mathbf{f}_3$  so they have large inner products  $\mathbf{f}_1^\top \mathbf{f}_3$  and  $\mathbf{f}_2^\top \mathbf{f}_3$ . This would mean that  $\mathbf{f}_1^\top \mathbf{f}_2$  would also be large, so  $\mathbf{f}_1$  would be strongly connected to  $\mathbf{f}_2$  under the BigClam model. However, some graphs, like bipartite graphs, do not exhibit this triangle inequality type behavior. For a rigorous treatment, see Section ?? and Appendix ?. Next, we build the IeClam decoder, that allows decoding any graph, that may have bipartite components without being limited by a triangle inequality-type behavior.

Our *Inclusive Exclusive Cluster Affiliation Model* (IeClam) can be extended from BigClam by replacing the inner product in the non-edge probability (??) by the more expressive *Lorentz inner product*, which does not enforce a triangle inequality-type behavior. For that, we extend the affiliation space (AS) to be  $\mathbb{R}^{2C}$ , with two types of communities. The first  $C$  axes are called the *inclusive communities*, and their corresponding features are called *inclusive affiliation features* (IAF), denoted by  $\mathbf{t} \in \mathbb{R}^C$ . The last  $C$  axes are called the *exclusive communities*, with *exclusive affiliation features* (EAF), denoted by  $\mathbf{s} \in \mathbb{R}^C$ .<sup>1</sup> We define the concatenated affiliation feature by  $\mathbf{f} = (\mathbf{t}, \mathbf{s}) \in \mathbb{R}^{2C}$ . Given a sequence of affiliation features  $\mathbf{F} = \{\mathbf{f}_n\}_{n=1}^N \in \mathbb{R}^{N \times 2C}$ , IeClam defines the probability of a single edge by

$$P(n \sim m | \mathbf{f}_n, \mathbf{f}_m) = 1 - \exp(-\mathbf{t}_n^\top \mathbf{t}_m + \mathbf{s}_n^\top \mathbf{s}_m) \\ = 1 - \exp(-\mathbf{f}_n^\top \mathbf{L} \mathbf{f}_m), \quad (7)$$

where  $\mathbf{L} = \text{diag}(1, \dots, 1, -1, \dots, -1)$ . The bilinear form  $(\mathbf{u}, \mathbf{v}) \mapsto \mathbf{u}^\top \mathbf{L} \mathbf{v}$  is called the *Lorentz inner product*, and

<sup>1</sup>More generally, one can define a different number of inclusive and exclusive communities.

has its roots in special relativity. See Appendix ?? for more details on the Lorentz inner product.

Note that  $\mathbf{L}$  is not positive-definite, so it does not actually define an inner product. Moreover,  $\mathbf{f}_n^\top \mathbf{L} \mathbf{f}_m$  can be negative even if  $\mathbf{f}_n, \mathbf{f}_m \in \mathbb{R}_+^C$ . To guarantee that (??) defines a proper probability between 0 and 1, we limit the affiliation space as follows.

**Definition 1.** A cone of non-negativity is a subset  $\mathcal{C}$  of  $\mathbb{R}^{2C}$  such that for every  $\mathbf{f}, \mathbf{g} \in \mathcal{C}$  we have  $\mathbf{f}^\top \mathbf{L} \mathbf{g} \geq 0$ .

If we limit the affiliation space to be a cone of non-negativity, then IeClam gives well defined probabilities in  $[0, 1]$ . In our experiments, we restrict ourselves to the following simple construction of a cone of non-negativity, noting that it is not the only possible construction.

**Definition 2.** The pairwise cone  $\mathcal{T}$  is defined to be the set of affiliation features  $\mathbf{f} = (\mathbf{t}, \mathbf{s}) \in \mathbb{R}^{2C}$  such that for every  $c \in [C]$  we have  $-t^c \leq s^c \leq t^c$ .

It is easy to see that  $\mathcal{T}$  is a cone of non-negativity. Indeed, for any  $\mathbf{f}_1 = (\mathbf{t}_1, \mathbf{s}_1), \mathbf{f}_2 = (\mathbf{t}_2, \mathbf{s}_2) \in \mathcal{T}$ , we have

$$\forall c \in [C] : t_1^c t_2^c - s_1^c s_2^c \geq 0,$$

so

$$\mathbf{f}_1^\top \mathbf{L} \mathbf{f}_2 = \sum_{c=1}^C t_1^c t_2^c - s_1^c s_2^c \geq 0.$$

As opposed to the BigClam decoder, the IeClam model can approximate a bipartite graph with a small number of communities. Namely, a bipartite graph with two (disjoint) sides  $\mathcal{U}, \mathcal{V} \subset [N]$  and constant probability for edges between  $\mathcal{U}$  and  $\mathcal{V}$  can be represented using  $C = 1$ . Here, all of the nodes in  $\mathcal{U}$  are encoded to  $(a, a)$ , and all of the nodes of  $\mathcal{V}$  are encoded to  $(a, -a)$ , for some  $a \geq 0$ . This gives zero probability for edges within each part, and probability  $1 - e^{-2a^2}$  for edges between the parts.

**Remark 3.** The above construction gives an interpretation for each pair of axes  $(t^c, s^c)$  in  $\mathcal{T}$  as a generalized community which can model anything between a clique and a bipartite component.

Given AFs  $\mathbf{F}$  in a cone, IeClam is seen as a decoder, by decoding  $\mathbf{F}$  into the random graph  $G(\mathbf{F})$  with node set  $[N]$  and independent Bernoulli distributed edges with probabilities  $\mathbf{P} = \{P(n \sim m | \mathbf{f}_n, \mathbf{f}_m)\}_{n,m=1}^N$ .

For affiliation features  $\mathbf{F} \in \mathcal{T}^N$ , the probability of the graph  $([N], E)$  given  $\mathbf{F}$  of IeClam is

$$P(E | \mathbf{F}) = \prod_{n \in [N]} \left( \sqrt{\prod_{m \in \mathcal{N}(n)} (1 - e^{-\mathbf{f}_n^\top \mathbf{L} \mathbf{f}_m}) \prod_{m \notin \mathcal{N}(n)} e^{-\mathbf{f}_n^\top \mathbf{L} \mathbf{f}_m}} \right). \quad (8)$$

Like BigClam, IeClam is optimized by maximizing the log likelihood with gradient descent

$$2l(\mathbf{F}) = \sum_{n \in [N]} \left( \sum_{m \in \mathcal{N}(n)} \log(1 - e^{-\mathbf{f}_n^\top \mathbf{L} \mathbf{f}_m}) - \sum_{m \notin \mathcal{N}(n)} \mathbf{f}_n^\top \mathbf{L} \mathbf{f}_m \right) \quad (9)$$

This loss can be efficiently implemented on sparse graph by the formulation

$$2l(\mathbf{F}) = \sum_{n \in [N]} \left( \sum_{m \in \mathcal{N}(n)} \log(e^{\mathbf{f}_n^\top \mathbf{L} \mathbf{f}_m} - 1) - \mathbf{f}_n^\top \mathbf{L} \sum_{n \in [N]} \mathbf{f}_m + \mathbf{f}_n^\top \mathbf{L} \mathbf{f}_n \right). \quad (10)$$

The gradient of the loss for node  $n$  is

$$\nabla_{\mathbf{f}_n} l = \mathbf{L} \left( \sum_{m \in \mathcal{N}(n)} \mathbf{f}_m (1 - e^{-\mathbf{f}_n^\top \mathbf{L} \mathbf{f}_m})^{-1} - \sum_{n \in [N]} \mathbf{f}_m + \mathbf{f}_n \right). \quad (11)$$

Notice that all of the calculations are the same as BigClam, up to replacing the dot product by the Lorenz inner product.

## 2.4 Community Affiliation Models With Prior

BigClam and IeClam are not generative graph models. Indeed, these methods only fit a conditional probability of the graph, conditioned on the AF values, but the methods do not learn the probability of the AFs over the affiliation space. Hence, the total probability of  $E \wedge \mathbf{F}$  is not defined.

To extend IeClam (and similarly BigClam) into probabilistic generative models, we define a prior probability distribution over the affiliation cone space  $\mathcal{C} \subset \mathbb{R}^{2C}$ , with probability density function  $p : \mathbb{R}^{2C} \rightarrow [0, \infty)$  supported on  $\mathcal{C}$ . Using Bayes law, we now obtain the joint probability  $P(E \wedge \mathbf{F})$  of the edges and community affiliation features via the probability density function

$$p(E, \mathbf{F}) = P(E|\mathbf{F})p(\mathbf{F}).$$

We assume that the prior probabilities of all nodes are independent, namely,

$$p(\mathbf{F}) = \prod_{n \in [N]} p(\mathbf{f}_n).$$

Hence, the probability densities that a dyad  $(n, m)$  is an edge or non-edge are

$$p(n \sim m, \mathbf{f}_n, \mathbf{f}_m) = p(\mathbf{f}_n)p(\mathbf{f}_m)(1 - e^{-\mathbf{f}_n^\top \mathbf{L} \mathbf{f}_m}),$$

$$p(\neg(n \sim m), \mathbf{f}_n, \mathbf{f}_m) = p(\mathbf{f}_n)p(\mathbf{f}_m)e^{-\mathbf{f}_n^\top \mathbf{L} \mathbf{f}_m}.$$

As before, we assume that the probabilities of different edges are independent, which gives

$$p(E, \mathbf{F}) = \prod_{n \in [N]} p(\mathbf{f}_n) \sqrt{\prod_{m \in \mathcal{N}(n)} P(n \sim m | \mathbf{F}_m) \prod_{m \notin \mathcal{N}(n)} P(\neg(n \sim m) | \mathbf{f}_n, \mathbf{f}_m)}. \quad (12)$$

Now, the log likelihood loss is

$$l(\mathbf{F}) = \sum_{n \in [N]} \left( \log(p(\mathbf{f}_n)) + \frac{1}{2} \left( \sum_{m \in \mathcal{N}(n)} \log(e^{\mathbf{f}_n^\top \mathbf{L} \mathbf{f}_m} - 1) - \mathbf{f}_n^\top \mathbf{L} \sum_{n \in [N]} \mathbf{f}_m + \mathbf{f}_n^\top \mathbf{L} \mathbf{f}_n \right) \right). \quad (13)$$

We call this extension of IeClam PieClam (Prior Inclusive Exclusive Cluster Affiliation Model). We similarly extend

BigClam to PClam (Prior Cluster Affiliation Model) by replacing  $\mathbf{L}$  in (??) with the identity matrix.

Observe that the PieClam loss is similar to the IeClam loss only, with the addition of the prior, acting as a per node regularization term. The prior attracts all nodes to areas of higher probability during the optimization dynamics.

In order to sample from the above generative models, we first sample features  $\{\mathbf{f}_n\}_{n \in [N]}$  according to  $p$ , and then connect them using either the BigClam or IeClam conditional probability. To model the prior in practice, we use *realNVP*, which is a *normalizing flow* neural network model (?). For more details on normalizing flows, see Appendix ??.

**PieClam for graphs with node features.** So far, we have used only the topology of the graph, not considering node features. We extend PieClam (and PClam) to graph-signals  $([N], E, \mathbf{X})$  as follows. Concatenate the feature space of  $\mathbf{X}$  with the affiliation space, and learn the prior on this combined space. This only affects the prior  $p$ . The conditional edge probabilities are defined only in terms of the affiliation features, as before.

## 3 Universality of PieClam and IeClam

In this section, we define the universality of graph autoencoders, and prove that IeClam and PieClam are universal, while BigClam and PClam are not. The motivation behind the universality definition is that we would like to uniformly choose the dimension of the code space, such that every graph can be approximated by decoding some points in this fixed code space. Namely, we would like one *universal* decoder that works for all graph, as opposed to choosing the dimension of the code space depending on the graph.

### 3.1 General Graph Autoencoders

Next, we define a general decoder that defines edge probabilities by operating on pairs of points in a code space.

**Definition 4.** A pairwise decoder over the code spaces  $\mathbb{R}^M$  is a mapping  $D_M : \mathbb{R}^{2M} \rightarrow [0, 1]$ . Given  $N$  points in the code space  $\mathbf{z} = \{z_n \in \mathbb{R}^M\}_{n=1}^N$ , the decoded graph  $G_N(\mathbf{z})$  is the weighted graph with adjacency matrix

$$\mathbf{D}_M(\mathbf{z}) = (D_M(z_n, z_k))_{n,k=1}^N.$$

Clam models are special cases of pairwise decoders.

### 3.2 Log Cut Distance

Our definition of universality has the following form: for every error tolerance  $\epsilon > 0$ , there is a choice of the dimension  $M(\epsilon)$  of the code space such that every graph can be approximated up to error  $\epsilon$  by decoding some points in this space. To formalize the “up to error  $\epsilon$ ” statement, we present in this section a new graph similarity measure which we call the log cut distance. Our construction is based on a well-known graph similarity measure called the cut norm.

**Definition 5.** The cut norm of a matrix  $\mathbf{X} \in \mathbb{R}^{N \times N}$  is defined to be

$$\|\mathbf{X}\|_{\square} := \frac{1}{N^2} \sup_{\mathcal{U}, \mathcal{V} \subset [N]} \left| \sum_{i \in \mathcal{U}} \sum_{j \in \mathcal{V}} x_{i,j} \right|. \quad (14)$$

The cut metric  $\|\mathbf{A} - \mathbf{B}\|_{\square}$  between two adjacency matrices  $\mathbf{A}$  and  $\mathbf{B}$  is interpreted as the difference between the edge densities of  $\mathbf{A}$  and  $\mathbf{B}$  on the block  $\mathcal{U} \times \mathcal{V}$  on which their edge densities are the most different.

The following graph similarity measure modifies the cut norm, making it appropriate for graphs with random edges over a fixed node set.

**Definition 6.** Given two random graphs over the nodes set  $[N]$ , with independently Bernoulli distributed edges, with probabilities  $\mathbf{P} = \{p_{n,m}\}_{n,m \in [N]}$  and  $\mathbf{Q} = \{q_{n,m}\}_{n,m \in [N]}$  respectively, their log cut distance is defined to be

$$D_{\square}(\mathbf{P}||\mathbf{Q}) := \inf_{0 < e, d \leq 1} \left( e + d + \frac{1}{N^2} \sup_{\mathcal{U}, \mathcal{V} \subset [N]} \left| \log \left( \prod_{n \in \mathcal{U}} \prod_{m \in \mathcal{V}} \frac{1 - (1 - e)p_{n,m}}{1 - (1 - d)q_{n,m}} \right) \right| \right). \quad (15)$$

The second term in (??) is the cut distance  $\|\tilde{\mathbf{P}} - \tilde{\mathbf{Q}}\|_{\square}$  between the matrix  $\tilde{\mathbf{P}}$  with entries

$$\tilde{p}_{n,m} = -\log(1 - (1 - e)p_{n,m})$$

and the matrix  $\tilde{\mathbf{Q}}$  with entries

$$\tilde{q}_{n,m} = -\log(1 - (1 - d)q_{n,m}).$$

Namely, the cut distance between the log likelihoods of non-edges. The parameters  $e$  and  $d$  make the  $[0, 1]$ -valued probabilities valid inputs to the log. The goal of  $e, d$  is to regularize the probability of the edges, where higher regularization is penalized via the additive term  $e + d$  in (??).

For each choice of a cut  $\mathcal{U}, \mathcal{V} \subset [N]$ , the term

$$\frac{1}{N^2} \log \left( \prod_{n \in \mathcal{U}} \prod_{m \in \mathcal{V}} \frac{1 - (1 - e)p_{n,m}}{1 - (1 - d)q_{n,m}} \right) \quad (16)$$

is somewhat similar in structure to an un-normalized KL divergence, or distance of log likelihoods, between the non-edge probabilities of the graphs  $\mathbf{P}$  and  $\mathbf{Q}$  over the dyads between  $\mathcal{U}$  and  $\mathcal{V}$ . Here, “un-normalized” means that the dyads are drawn uniformly with probabilities  $1/N^2$ , but the sum of probabilities is  $|\mathcal{U}| \cdot |\mathcal{V}|/N^2$  and not 1. The un-normalized uniform distribution discourages the supremum inside the definition of  $D_{\square}$  from choosing small blocks for maximizing (??). Note that normalized uniform distributions would lead  $D_{\square}$  to choose small blocks, which do not reflect meaningful empirical estimates of the edge statistics (the edge densities of small blocks would not be interpretable as expected number of edges). To conclude,  $D_{\square}(\mathbf{P}||\mathbf{Q})$  is interpreted as the maximal divergence between  $\mathbf{P}$  and  $\mathbf{Q}$  over all blocks, up to the best regularizers  $e, d$ .

In our analysis we compute the log cut distance between the random decoded graph  $\mathbf{P}$  and the deterministic target graph  $\mathbf{A}$ . While  $\mathbf{A}$  has edge probabilities in  $\{0, 1\}$ ,  $\mathbf{P}$  has edge probabilities in  $[0, 1)$  for Clam models. Therefore, we only require regularization for  $\mathbf{A}$ . We hence consider the following modified version of Definition ??.

**Definition 7.** Given an unweighted graph with adjacency matrix  $\mathbf{A} \in \{0, 1\}^{N \times N}$  and a random graph over the nodes

set  $[N]$ , with independently Bernoulli distributed edges with probabilities  $\mathbf{P} = \{0 \leq p_{n,m} < 1\}_{n,m \in [N]}$ , the log cut distance between  $\mathbf{P}$  and  $\mathbf{A}$  is defined to be

$$D_{\square}(\mathbf{P}||\mathbf{A}) :=$$

$$\inf_{0 < d \leq 1} \left( d + \frac{1}{N^2} \sup_{\mathcal{U}, \mathcal{V} \subset [N]} \left| \log \left( \prod_{n \in \mathcal{U}} \prod_{m \in \mathcal{V}} \frac{1 - p_{n,m}}{1 - (1 - d)a_{n,m}} \right) \right| \right).$$

Lastly, in case both  $\mathbf{P}$  and  $\mathbf{Q}$  are  $[0, 1)$ -valued, a simple version of the log cut distance is (??) with the choice  $e = d = 0$ , namely,

$$D_{\square}^0(\mathbf{P}||\mathbf{Q}) := \frac{1}{N^2} \sup_{\mathcal{U}, \mathcal{V} \subset [N]} \left| \log \left( \prod_{n \in \mathcal{U}} \prod_{m \in \mathcal{V}} \frac{1 - p_{n,m}}{1 - q_{n,m}} \right) \right|. \quad (17)$$

### 3.3 Universal Graph Autoencoders

We are now ready to define the universality of general pairwise autoencoders. Motivated by the fact that a Clam autoencoder is actually a family of autoencoders, parameterized by the number of communities, we also define general pairwise decoders as families.

**Definition 8.** A family of code spaces  $\mathbb{R}^M$  and corresponding pairwise decoders  $D_M : \mathbb{R}^{2M} \rightarrow [0, 1]$ , parametrized by  $M \in \mathbb{N}$ , is called universal if for every  $\epsilon > 0$  there is  $M \in \mathbb{N}$  (which depends only on  $\epsilon$ ) such that for every  $N \in \mathbb{N}$  and every graph with adjacency matrix  $\mathbf{A}$  and  $N$  nodes there are  $N$  points in the code space  $\{z_n \in \mathbb{R}^M\}_{n=1}^N$  such that

$$D_{\square}(D_M(\mathbf{z})||\mathbf{A}) < \epsilon.$$

### 3.4 BigClam and PClam are Not Universal

We now show that BigClam (and hence also PClam) is not a universal autoencoder since it cannot approximate bipartite graphs. Consider the bipartite graph  $\mathbf{B}$  with  $N$  nodes at each part, and probability  $1 - e^{-a^2}$  for an edge between the two parts, and 0 within each part. Since in this case all probabilities are less than 1, we can use (??) as the definition of the log cut distance. The analysis for Definition ?? extends naturally.

Let  $\mathbf{P}$  be a decoded BigClam graph. Our goal is to show that there is no way to make  $D_{\square}^0(\mathbf{P}||\mathbf{Q})$  small by choosing the dimension  $C$  uniformly with respect to  $N$ . In fact, we will show BigClam cannot approximate a bipartite graph at all.<sup>2</sup>

**Claim 9.** Under the above construction,

$$D_{\square}^0(\mathbf{P}||\mathbf{B}) \geq \frac{a^2}{16}.$$

As a result, BigClam is not a universal autoencoder.

The proof is given in Appendix ?? . We note that one can similarly show that BigClam is not universal also with respect to the log cut distance of Definition ??.

<sup>2</sup>In Appendix ?? we show that one can approximate a bipartite graph of  $2N$  nodes using  $C = N^2$  classes in BigClam if the model ignores self-loops.

### 3.5 Universality of IeClam and PieClam

We are now ready to show that IeClam (and hence also PieClam) is a universal autoencoder. The proofs of the following two theorems are in Appendix ?? and ??.

We give two versions for the universality result. The first is without a cone restriction, and requires a relatively small number of communities for the given error tolerance. The corresponding decoder produces edge weights that can be negative. The second theorem is restricted to the pairwise cone of non-negativity, and has pessimistic asymptotics for the required number of communities given an error tolerance. This decoder is guaranteed to produce proper edge probabilities in  $[0, 1]$ .

**Theorem 10.** *For every epsilon  $\epsilon > 0$ , every  $N \in \mathbb{N}$ , and every adjacency matrix  $\mathbf{A} \in [0, 1]^{N \times N}$ , there are  $N$  affiliation features  $\mathbf{F} \in \mathbb{R}^{2K}$  of dimension  $K = -9 \log(\epsilon/2)^2 / \epsilon^2$  such that the corresponding IeClam model  $\mathbf{P} = \{P(n \sim m | \mathbf{f}_n, \mathbf{f}_m)\}_{n,m=1}^N$  satisfies*

$$D_{\square}(\mathbf{P} || \mathbf{A}) < \epsilon.$$

Here, the log cut distance is from Definition ?. As a result, IeClam and PieClam are universal autoencoders with code space  $\mathbb{R}^{2K}$ .

**Theorem 11.** *For every epsilon  $\epsilon > 0$ , every  $N \in \mathbb{N}$ , and every adjacency matrix  $\mathbf{A} \in [0, 1]^{N \times N}$ , there are  $N$  affiliation features  $\mathbf{F}$  in the cone of pairwise non-negativity  $\mathcal{T} \subset \mathbb{R}^{2C}$  of dimension  $C = 2^4 \lceil -\log(\epsilon/2)^2 / \epsilon^2 \rceil$  such that the corresponding IeClam model  $\mathbf{P} = \{P(n \sim m | \mathbf{f}_n, \mathbf{f}_m)\}_{n,m=1}^N$  satisfies*

$$D_{\square}(\mathbf{P} || \mathbf{A}) < \epsilon.$$

Here, the log cut distance is from Definition ?. As a result, IeClam and PieClam are universal autoencoders with code space  $\mathcal{T}$ .

## 4 Experiments

### 4.1 Reconstructing Synthetic Priors

We consider a ground-truth synthetic prior  $p : \mathbb{R}^C \rightarrow [0, \infty)$  in PClam. We sample  $N = 500$  points from the prior, decode the corresponding PClam graph, and sample a simple graph from the random Bernoulli edges. Then, given these sampled graph, we fit to it a PClam and model. In Figure ?? we compare the ground-truth prior to the reconstructed prior. We observe that even though our method is unsupervised, it still manages to capture the prior qualitatively well. More details are given in Appendix ??.

### 4.2 Reconstructing Synthetic SBMs

In Figures ?? and ?? we consider a synthetic SBM, and sample simple graph with  $N = 210$  nodes from it. We then fit a PClam and PieClam model to it. The SBM is not dominant diagonal, so it cannot be well approximated by the PClam model (which finds a nested community structure), while the PieClam model approximates it well qualitatively. Additional details are given in Appendix ??.

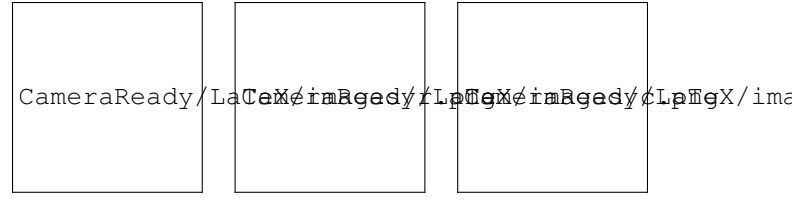


Figure 1: Left to right: Synthetic prior in an affiliation space of two inclusive communities. Reconstructed prior by PClam with normalizing flow. Reconstructed affiliation features by PClam.

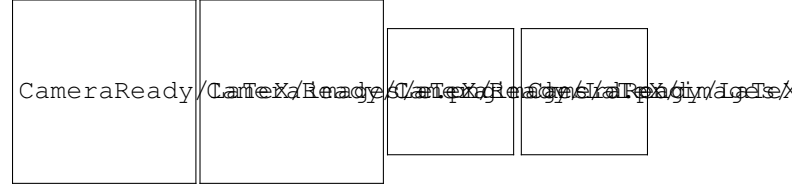


Figure 2: Left to right: Adjacency matrix sampled from SBM with three classes and 9 blocks. Adjacency matrix of the fitted PieClam graph, with two inclusive and two exclusive communities. Affiliation features of the PieClam matrix in  $\mathcal{T}$  projected to  $(t^1, s^1)$  and  $(t^2, s^2)$ .

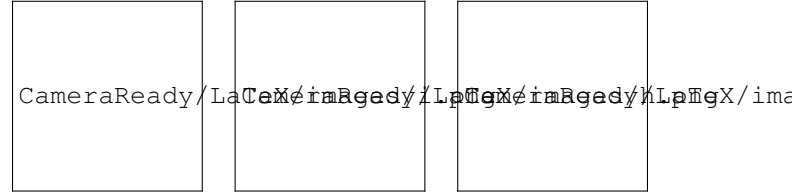


Figure 3: Left to right: Adjacency matrix sampled from SBM. Fitted PClam adjacency matrix based on two inclusive communities. Affiliations features of the PClam matrix.

### 4.3 Anomaly Detection

In unsupervised node anomaly detection, one is given a graph with node features, where some of the nodes are unknown anomalies. The goal is to detect these anomalous nodes, without supervising on any example of normal or anomalous nodes, by only using the structure of the graph and the node features. We use Clam models for node anomaly detection by fitting the Clam model to the graph and flagging nodes as anomalous if they satisfy the following different criteria.

- **(S) Star probability:** Given any Clam model, a node  $n$  is called anomalous if  $\prod_{m \in \mathcal{N}(n)} P(n \sim m | \mathbf{F}) < \delta$ .
- **(P) Prior probability:** Given any Clam model with prior, a node  $n$  is called anomalous if  $p(\mathbf{f}_n) < \delta$ .
- **(PS) Prior star probability:** Given any Clam model with prior, a node  $n$  is called anomalous if  $p(\mathbf{f}_n) \prod_{m \in \mathcal{N}(n)} P(n \sim m | \mathbf{F}) < \delta$ .

We reduce the dimension of the node features of the input graphs to 100 using truncated SVD, unless the dimension of

| Method         | Reddit       | Elliptic     | Photo        |
|----------------|--------------|--------------|--------------|
| (S)- IeClam    | <b>0.639</b> | 0.440        | <b>0.592</b> |
| (S) - PieClam  | 0.610        | 0.435        | 0.556        |
| (P) - PieClam  | 0.567        | <b>0.610</b> | 0.425        |
| (PS) - PieClam | *0.612       | <u>0.551</u> | 0.507        |
| (S) - BigClam  | <u>0.637</u> | 0.434        | <u>0.581</u> |
| DOMINANT       | 0.511        | 0.296        | 0.514        |
| AnomalyDAE     | 0.509        | *0.496       | 0.507        |
| OCGNN          | 0.525        | 0.258        | 0.531        |
| AEGIS          | 0.535        | 0.455        | 0.552        |
| GAAN           | 0.522        | 0.259        | 0.430        |
| TAM            | 0.606        | 0.404        | *0.568       |

Table 1: Comparison of Clam anomaly detectors with competing methods. First place in **boldface**, second with underline, third with \*star. We observe that our methods are first and second place on all datasets. Moreover, S- IeClam, PS-PieClam and S BigClam each beats the competing methods in two out of the three datasets. The accuracy metric is areas under curve (AUC).

the features is smaller than 100 in which case we only normalize them to have zero mean and standard deviation one. We use an affiliation space embedding dimension of 30 for **F** for PieClam and IeClam, and 24 for BigClam. Every Clam method starts with a random embedding **F**. Clam models with prior are trained with the following steps. (**F**- $t$ ): given a fixed prior  $p$ , optimize only **F** for  $t$  steps. ( $p$ - $t$ ): given a fixed embedding **F**, optimize only  $p$  for  $t$  steps. For regularization, in each iteration of  $p$  we add Gaussian noise with amplitude 0.01 to the affiliation features. We train PieClam with scheme **F**-500  $\rightarrow$   $p$ -1300  $\rightarrow$  **F**-500  $p$ -1300 with learning rate of  $2e^{-6}$  on **F** and  $1e^{-6}$  on  $p$ . For the models which only optimize **F** (IeClam and BigClam) we use the following configurations: We train IeClam on 2500 iterations with learning rate  $1e^{-6}$  using 30 communities. We train BIGClam on 2200 iterations with learning rate  $1e^{-6}$ . More details on hyper-parameters are given in Appendix ??.

In Table ?? we compare the performance of Clam methods to DOMINANT (?), AnomalyDAE (?), OCGNN (?), AEGIS (?), GAAN (?) and TAM (?) on the datasets Reddit, Elliptic, and Photo. The hyper-parameters of the competing methods are taken as the recommended values from the respective papers. The results are taken from Table 1 in (?). We observe that our methods are first and second place on all datasets. Moreover, S- IeClam, PS-PieClam and S BigClam each beats the competing methods in two out of the three datasets.

#### 4.4 Link Prediction

In supervised link prediction, one is given a graph for which some of the connections are unknown. The goal is to train the model using the known connections and predict the connectivity of the so called omitted dyads. In the following, we predict the probability of an edge using the conditional probability given in ??. Since the prior is not used for prediction, there is no added benefit in concatenating the node features to the affiliation features as we did in ?? and so the

| Method    | Squirrel    | Photo       | Texas       | JH55        |
|-----------|-------------|-------------|-------------|-------------|
| PieClam   | <b>98.8</b> | <b>98.4</b> | <b>86.4</b> | 95.6*       |
| BigClam   | <u>98.5</u> | 97.4*       | 78.8*       | 94.9        |
| VGAE      | 98.2        | 94.9        | 68.6        | 92.8        |
| GAT       | 98.0        | 97.3        | 68.5        | 94.3        |
| LINKX     | 98.1        | 97.0        | 75.8        | 93.4        |
| AA        | 97.1        | 97.4        | 53.1        | <u>96.1</u> |
| DisenLink | 98.3*       | <u>97.9</u> | <u>80.7</u> | <b>97.5</b> |

Table 2: Comparison of Clam link predictors with competing models. First place in **boldface**, second with underline, third with \*star

link prediction reported in this paper **only uses the graph structure**. After assigning probabilities to all of the omitted dyads, we calculate the AUC score of the classification. We use the experimental setting presented in (?). For every dataset, we omit 10% of the edges as a test set and find the hyperparameters by talking a validation set of another 5% for every hyperparameter configuration. In addition to omitting edges, we also omit 5 non-edges for every omitted edge. Finally, we run the model 10 times on the test set with the fitted hyperparameters. We run our experiments on the datasets Squirrel and Texas (?), Photo (?) and Facebook100's Johns Hopkins (?) and compare PieClam to the baselines AA (?), VGAE (?), GAT (?), LINKX (?) and Disenlink (?). The results are presented in Table ??, and the baseline results are taken from Table 2 in (?). An in depth explanation on the experimental setup and a table with confidence intervals is given in the Appendix.

## 5 Conclusion

We introduced PieClam, a new probabilistic graph generative model. PieClam models graphs via embedding the nodes into an inclusive and exclusive communities space, learning a prior distribution in this space, and decoding pairs of points in this space to edge probabilities, such that points are more likely to be connected the more inclusive communities and the less exclusive communities they share. We showed that PieClam is a universal autoencoder, able to approximate any graph, where the budget of parameters (the number of communities) can be predefined, irrespective of any property of specific graph, not even the number of nodes. Our experiments show that PieClam achieves competitive results when used in graph anomaly detection.

One limitation of PieClam is that, for attributed graphs, it only models the node features through the prior in the community affiliation space, but not via the conditional probabilities of the edges (given the community affiliations). Future work will deal with extending PieClam to also include the node (or edge) features in the edge conditional probabilities. Another limitation of our analysis is that the log cut distance is mainly appropriate for dense graphs. Future work will extend this metric to sparse graphs. This can be done, e.g., similarly to the sparse constructions in (?).

## Reproducibility Checklist

This paper:

- Includes a conceptual outline and/or pseudocode description of AI methods introduced **(yes/partial/no/NA)**
- Clearly delineates statements that are opinions, hypothesis, and speculation from objective facts and results **(yes/no)**
- Provides well marked pedagogical references for less-familiar readers to gain background necessary to replicate the paper **(yes/no)**

Does this paper make theoretical contributions? **(yes/no)**

If yes, please complete the list below.

- All assumptions and restrictions are stated clearly and formally. **(yes/partial/no)**
- All novel claims are stated formally (e.g., in theorem statements). **(yes/partial/no)**
- Proofs of all novel claims are included. **(yes/partial/no)**
- Proof sketches or intuitions are given for complex and/or novel results. **(yes/partial/no)**
- Appropriate citations to theoretical tools used are given. **(yes/partial/no)**
- All theoretical claims are demonstrated empirically to hold. **(yes/partial/no/NA)**
- All experimental code used to eliminate or disprove claims is included. **(yes/no/NA)**

Does this paper rely on one or more datasets? **(yes/no)**

If yes, please complete the list below.

- A motivation is given for why the experiments are conducted on the selected datasets **(yes/partial/no/NA)**
- All novel datasets introduced in this paper are included in a data appendix. **(yes/partial/no/NA)**
- All novel datasets introduced in this paper will be made publicly available upon publication of the paper with a license that allows free usage for research purposes. **(yes/partial/no/NA)**
- All datasets drawn from the existing literature (potentially including authors' own previously published work) are accompanied by appropriate citations. **(yes/no/NA)**
- All datasets drawn from the existing literature (potentially including authors' own previously published work) are publicly available. **(yes/partial/no/NA)**
- All datasets that are not publicly available are described in detail, with explanation why publicly available alternatives are not scientifically satisfying. **(yes/partial/no/NA)**

Does this paper include computational experiments? **(yes/no)**

If yes, please complete the list below.

- Any code required for pre-processing data is included in the appendix. **(yes/partial/no)**.
- All source code required for conducting and analyzing the experiments is included in a code appendix. **(yes/partial/no)**

- All source code required for conducting and analyzing the experiments will be made publicly available upon publication of the paper with a license that allows free usage for research purposes. **(yes/partial/no)**
- All source code implementing new methods have comments detailing the implementation, with references to the paper where each step comes from **(yes/partial/no)**
- If an algorithm depends on randomness, then the method used for setting seeds is described in a way sufficient to allow replication of results. **(yes/partial/no/NA)**
- This paper specifies the computing infrastructure used for running experiments (hardware and software), including GPU/CPU models; amount of memory; operating system; names and versions of relevant software libraries and frameworks. **(yes/partial/no)**
- This paper formally describes evaluation metrics used and explains the motivation for choosing these metrics. **(yes/partial/no)**
- This paper states the number of algorithm runs used to compute each reported result. **(yes/no)**
- Analysis of experiments goes beyond single-dimensional summaries of performance (e.g., average; median) to include measures of variation, confidence, or other distributional information. **(yes/no)**
- The significance of any improvement or decrease in performance is judged using appropriate statistical tests (e.g., Wilcoxon signed-rank). **(yes/partial/no)**
- This paper lists all final (hyper-)parameters used for each model/algorithm in the paper's experiments. **(yes/partial/no/NA)**
- This paper states the number and range of values tried per (hyper-) parameter during development of the paper, along with the criterion used for selecting the final parameter setting. **(yes/partial/no/NA)**



# Supplementary Material

## A Proofs

### A.1 Proof That BigClam Is Not Universal

In this subsection we Prove Claim ?? . Consider the bipartite graph  $\mathbf{B}$  with  $N$  nodes at each part, and probability  $1 - e^{-a^2}$  for an edge between the two parts, and 0 within each part. Consider (??) as the definition of the log cut distance. Let  $\mathbf{P}$  be a decoded BigClam graph from the affiliation features  $\mathbf{F}$ .

Denote by  $\tilde{\mathbf{P}}$  the matrix with entries

$$\tilde{p}_{n,m} = -\log(1 - p_{n,m}) = \mathbf{f}_n^\top \mathbf{f}_m,$$

and by  $\tilde{\mathbf{B}}$  the matrix with entries  $\tilde{b}_{n,m} = -\log(1 - b_{n,m})$ . We show that there is no way to make  $\|\tilde{\mathbf{P}} - \tilde{\mathbf{Q}}\|_\square$  small.

**Claim 12.** *Under the above construction,*

$$D_\square^0(\mathbf{P}||\mathbf{B}) \geq \frac{a^2}{16}.$$

As a result, BigClam is not a universal autoencoder.

*Proof.* Note that  $\tilde{b}_{n,m} = a^2$  if  $n, m$  are in opposite parts, and  $\tilde{b}_{n,m} = 0$  if  $n, m$  are on the same side. We index the nodes such that  $[N]$  is the first side of the graph, and  $[N] + N$  is the second side. For  $n \in [N]$  we denote  $\mathbf{q}_n = \mathbf{f}_n$  and  $\mathbf{y}_n = \mathbf{f}_{n+N}$ . Next, we use the identity

$$D_\square^0(\mathbf{P}||\mathbf{Q}) = \|\tilde{\mathbf{P}} - \tilde{\mathbf{B}}\|_\square,$$

and bound the right-hand-side from below.

First, by the definition of cut norm, for every  $\mathcal{U}, \mathcal{V} \subset [2N]$ ,

$$\|\tilde{\mathbf{P}} - \tilde{\mathbf{B}}\|_\square \geq \left| \frac{1}{4N^2} \sum_{n \in \mathcal{U}} \sum_{m \in \mathcal{V}} (\tilde{p}_{n,m} - \tilde{b}_{n,m}) \right|.$$

Hence, for  $\mathcal{U}_1 = \mathcal{V}_1 = [N]$ ,  $\mathcal{U}_2 = \mathcal{V}_2 = [N] + 1$ ,  $\mathcal{U}_3 = [N]$ ,  $\mathcal{V}_3 = [N] + N$ , and  $\mathcal{U}_4 = [N] + N$ ,  $\mathcal{V}_4 = [N]$ , we have

$$\begin{aligned} & \|\tilde{\mathbf{P}} - \tilde{\mathbf{B}}\|_\square \geq \\ & \frac{1}{16N^2} \sum_{j=1}^4 \left| \sum_{n \in \mathcal{U}_j} \sum_{m \in \mathcal{V}_j} (\tilde{p}_{n,m} - \tilde{b}_{n,m}) \right| \\ & = \frac{1}{16N^2} \sum_{n=1}^N \sum_{m=1}^N \mathbf{q}_n^\top \mathbf{q}_m + \frac{1}{16N^2} \sum_{n=1}^N \sum_{m=1}^N \mathbf{y}_n^\top \mathbf{y}_m \\ & \quad + \frac{1}{8N^2} \sum_{n=1}^N \sum_{m=1}^N (a^2 - \mathbf{q}_n^\top \mathbf{y}_m). \end{aligned} \quad (18)$$

Denote

$$\mathbf{q} = \frac{1}{N} \sum_{n=1}^N \mathbf{q}_n, \quad \mathbf{y} = \frac{1}{N} \sum_{n=1}^N \mathbf{y}_n.$$

With these notations, (??) can be written as

$$\begin{aligned} & \|\tilde{\mathbf{P}} - \tilde{\mathbf{B}}\|_\square \geq \\ & \frac{1}{16} \mathbf{q}^\top \mathbf{q} + \frac{1}{16} \mathbf{y}^\top \mathbf{y} + \frac{1}{8} (a^2 - \mathbf{q}^\top \mathbf{y}) \\ & = \frac{1}{16} (a^2 + (\mathbf{q}^\top - \mathbf{y}^\top)(\mathbf{q} - \mathbf{y})) \geq \frac{a^2}{16}. \end{aligned}$$

□

We note that one can similarly show that BigClam is not universal also with respect to the log cut distance of Definition ??.

### A.2 Proof That a Monotonous Inner Product Decoder Is Not Universal in Log Cut

In this subsection we Prove Claim ?? . Consider the bipartite graph  $\mathbf{B}$  with  $N$  nodes at each part, and probability 1 for an edge between the two parts, and 0 within each part. Consider Definition ?? as the definition of the log cut distance. Let  $g$  such that  $g : \mathbb{R} \rightarrow [0, 1]$  and  $g$  is ascending. Let  $\{\mathbf{f}_n\}_{n=1}^{2N}$  be some affiliation vector set for  $\mathbf{B}$  and let  $\mathbf{P}$  be the matrix such that  $p_{nm} = \Pr(n \sim m \mid \mathbf{f}_n, \mathbf{f}_m) = g(\mathbf{f}_n^\top \mathbf{f}_m)$  be a random graph reconstruction of  $\mathbf{B}$ . Note that BigClam is a special case of this construction.

Denote by  $\tilde{\mathbf{P}}$  the matrix with entries

$$\tilde{p}_{n,m} = -\log(1 - p_{n,m}) = -\log(1 - g(\mathbf{f}_n^\top \mathbf{f}_m)),$$

Let  $d \in [0, 1]$  and denote by  $\tilde{\mathbf{B}}^d$  the matrix with entries  $\tilde{b}_{n,m}^d = -\log(1 - (1 - d)b_{n,m})$ . **[Ron: What is B? If it's the same as in my construction then why does your bound not depend on a?]** In here I'm using definition ?? for the log cut and so the bipartite does not depend on  $a$ . it's just a regular bipartite:  $b_{mn} = 0$  if they are in the same part and 1 if they are in different parts

**Claim 13.** *Under the above construction,*

$$D_\square(\mathbf{P}||\mathbf{B}) \geq 4 \frac{1}{16}.$$

As a result, any autoencoder that uses a monotonous ascending function of the inner product between affiliation features for decoding is not a universal autoencoder.

*Proof.* Note that  $\tilde{b}_{n,m}^d = -\log(d)$  if  $n, m$  are in opposite sides, and  $\tilde{b}_{n,m}^d = 0$  if  $n, m$  are on the same side. We index the nodes such that  $[N]$  is the first side of the graph  $\mathcal{C}_1$ , and  $[N] + N$  is the second side  $\mathcal{C}_2$ . For  $n \in [N]$  we denote  $\mathbf{q}_n = \mathbf{f}_n$  and  $\mathbf{y}_n = \mathbf{f}_{n+N}$ . Next, we use the definition

$$D_\square(\mathbf{P}||\mathbf{B}) = \inf_{d \in [0,1]} (d + \|\tilde{\mathbf{P}} - \tilde{\mathbf{B}}^d\|_\square),$$

and bound the right-hand-side for our arbitrarily selected  $d$  from below:

First, by the definition of cut norm, for every  $\mathcal{U}, \mathcal{V} \subset [2N]$ ,

$$\|\tilde{\mathbf{P}} - \tilde{\mathbf{B}}^d\|_\square \geq \left| \frac{1}{4N^2} \sum_{n \in \mathcal{U}} \sum_{m \in \mathcal{V}} (\tilde{p}_{n,m} - \tilde{b}_{n,m}^d) \right|.$$

Hence, for  $\mathcal{U}_1 = \mathcal{V}_1 = \mathcal{C}_1$ ,  $\mathcal{U}_2 = \mathcal{V}_2 = \mathcal{C}_2$ ,  $\mathcal{U}_3 = \mathcal{C}_1$ ,  $\mathcal{V}_3 = \mathcal{C}_2$ , and  $\mathcal{U}_4 = \mathcal{C}_2$ ,  $\mathcal{V}_4 = \mathcal{C}_1$ , we have

$$\begin{aligned} & \|\tilde{\mathbf{P}} - \tilde{\mathbf{B}}^d\|_\square \geq \\ & \frac{1}{16N^2} \sum_{j=1}^4 \left| \sum_{n \in \mathcal{U}_j} \sum_{m \in \mathcal{V}_j} (\tilde{p}_{n,m} - \tilde{b}_{n,m}^d) \right| \\ & = \frac{1}{16N^2} \left( \sum_{n=1}^N \sum_{m=1}^N -\log(1 - g(\mathbf{q}_n^\top \mathbf{q}_m)) + \right. \end{aligned}$$

$$+ \sum_{n=1}^N \sum_{m=1}^N -\log(1 - g(\mathbf{y}_n^\top \mathbf{y}_m)) \quad (19)$$

$$+ 2 \left| \sum_{n=1}^N \sum_{m=1}^N (-\log(d) + \log(1 - g(\mathbf{q}_n^\top \mathbf{y}_m)) \right|. \quad (20)$$

Denote

$$\begin{aligned} \xi : [0, 1] &\rightarrow \mathbb{R}, \quad \xi(x) = -\log(1 - g(x)) \\ \langle \mathcal{C}_i, \mathcal{C}_i \rangle &= \frac{1}{N^2} \sum_{n \in \mathcal{C}_i} \sum_{m \in \mathcal{C}_i} \xi(\mathbf{f}_n^\top \mathbf{f}_m) \end{aligned}$$

Notice that  $\xi \geq 0$  and is ascending, and so, by matching each of the elements in the sums for  $\langle \mathcal{C}_i, \mathcal{C}_j \rangle$  we can easily see that (((DAN: easily see is because WLG  $\mathbf{a}^\top \mathbf{b} \leq \mathbf{a}^\top \mathbf{a}$  so  $\xi(\mathbf{a}^\top \mathbf{b}) \leq \xi(\mathbf{a}^\top \mathbf{a}) \leq \xi(\mathbf{a}^\top \mathbf{a}) + \xi(\mathbf{b}^\top \mathbf{b})$ ))))

$$\begin{aligned} \langle \mathcal{C}_1, \mathcal{C}_2 \rangle &\leq \langle \mathcal{C}_1, \mathcal{C}_1 \rangle + \langle \mathcal{C}_2, \mathcal{C}_2 \rangle \\ \langle \mathcal{C}_1, \mathcal{C}_2 \rangle &= \langle \mathcal{C}_2, \mathcal{C}_1 \rangle \end{aligned} \quad (21)$$

With these notations, (??) can be written as

$$\|\tilde{P} - \tilde{B}^d\|_{\square} \geq \frac{1}{16} \left( \langle \mathcal{C}_1 \mathcal{C}_1 \rangle + \langle \mathcal{C}_2 \mathcal{C}_2 \rangle + 2 \left| -\log(d) - \langle \mathcal{C}_1 \mathcal{C}_2 \rangle \right| \right)$$

**Case 1:** If  $-\log(d) \geq \langle \mathcal{C}_1 \mathcal{C}_2 \rangle$ :

$$\begin{aligned} \|\tilde{P} - \tilde{B}^d\|_{\square} &\geq \frac{1}{16} \left( \langle \mathcal{C}_1 \mathcal{C}_1 \rangle + \langle \mathcal{C}_2 \mathcal{C}_2 \rangle - 2 \log(d) - 2 \langle \mathcal{C}_1 \mathcal{C}_2 \rangle \right) \geq \\ &\frac{1}{16} \left( -2 \log(d) - \langle \mathcal{C}_1 \mathcal{C}_2 \rangle \right) \geq \\ &\frac{1}{16} \left( -\log(d) + \langle \mathcal{C}_1 \mathcal{C}_2 \rangle - \langle \mathcal{C}_1 \mathcal{C}_2 \rangle \right) = \\ &-\frac{1}{16} \log(d) \end{aligned}$$

**Case 2:** If  $-\log(d) \leq \langle \mathcal{C}_1 \mathcal{C}_2 \rangle$ :

$$\begin{aligned} \|\tilde{P} - \tilde{B}^d\|_{\square} &\geq \frac{1}{16} \left( \langle \mathcal{C}_1 \mathcal{C}_1 \rangle + \langle \mathcal{C}_2 \mathcal{C}_2 \rangle + 2 \langle \mathcal{C}_1 \mathcal{C}_2 \rangle + 2 \log(d) \right) \geq \\ &\frac{1}{16} \left( \langle \mathcal{C}_1 \mathcal{C}_1 \rangle + \langle \mathcal{C}_2 \mathcal{C}_2 \rangle \right) \geq \\ &-\frac{1}{16} \log(d) \end{aligned}$$

In both cases, by using Equation ?? we see that

$$\begin{aligned} d + \|\tilde{P} - \tilde{B}^d\|_{\square} &\geq \\ d - \frac{1}{16} \log(d) \end{aligned}$$

For  $d \in [0, 1]$ , this function has one minimum at  $d = 1/16$  and so we conclude that for any  $d \in [0, 1]$

$$d + \|\tilde{P} - \tilde{B}^d\|_{\square} \geq 4 \frac{1}{16}$$

This inequality holds for any choice of  $d$  and so we conclude

$$D_{\square}^0(\mathbf{P} || \mathbf{B}) \geq 4 \frac{1}{16}.$$

□

**Note:** If  $g$  is descending, use a graph with two fully connected communities of the same size as the two problems are equivalent.

### A.3 BigClam With No Self Loops Approximating Bipartite Graphs

Consider the above bipartite graph  $\mathbf{B}$  with  $N$  nodes at each part, and probability  $1 - e^{-a^2}$  for an edge between the two parts, and 0 within each part. If we redefine the BigClam decoder to have no self-loops, namely,  $\mathbf{P}$  has entries  $p_{n,m} = P(n \sim m | \mathbf{f}_n, \mathbf{f}_m)$  for  $n \neq m$ , and  $p_{n,m} = 0$  for  $n = m$ , then one can obtain a bipartite  $\mathbf{P}$  with  $C = N^2$  communities as follows.

In the following analysis, an addition or multiplication of a set by a scalar is defined to be the addition or multiplication of every element in the set by this scalar. Encode each node  $n \in [N]$  in part 1 to  $\mathbf{f}_n$  with  $f_n^c = a$  for  $c \in [N] + n(N-1)$  and  $f_n^c = 0$  otherwise. Encode every node  $n \in [N] + N$  from side 2 to  $\mathbf{f}_n$  with  $f_n^c = a$  for  $c \in N([N] - 1) + n$  and  $f_n^c = 0$  otherwise. It is easy to see that the corresponding  $\mathbf{P}$  is bipartite with edge probability between the parts being  $1 - e^{-a^2}$ .

### A.4 Proof of the Universality of PieClam

The proof is based on a version of the weak regularity lemma for intersecting communities. While the standard weak regularity lemma (??) partitions the graph into disjoint communities, it is well known that allowing the communities to overlap allows using much less communities, which improves the asymptotics of the approximation. The regularity lemma was used in the context of graph machine learning in (??). To formalize the relevant version of the weak regularity theorem for our analysis, we first need to cite a definition from (?).

**Definition 14.** A (hard) intersecting community graph (ICG) with  $N$  nodes and  $K$  communities is a matrix  $\mathbf{C} \in \mathbb{R}^{N \times N}$  of the following form. There exist  $K$  column vectors  $\mathbf{Q} = (\mathbf{q}_k \in \{0, 1\}^N)_{k=1}^K \in \{0, 1\}^{N \times K}$  and  $k$  coefficients  $\mathbf{r} = (r_k \in \mathbb{R})_{k=1}^K \in \mathbb{R}^K$  such that

$$\mathbf{C} = \mathbf{Q} \text{diag}(\mathbf{r}) \mathbf{Q}^\top.$$

The following is a special case of the weak regularity lemma from (?), up to the small modification to the adjacency matrix, allowing it to have values in  $[0, R]$  instead of  $[0, 1]$ .

**Theorem 15.** Let  $\mathbf{A} \in [0, R]^{N \times N}$  be an adjacency matrix of a graph with  $N$  nodes. Let  $\epsilon > 0$ . Denote  $K = \frac{9R^2}{4\epsilon^2}$ . Then, there exists a hard ICG  $\mathbf{C}$  with  $K$  communities such that

$$\|\mathbf{A} - \mathbf{C}\|_{\square} \leq \epsilon. \quad (22)$$

*Proof of Theorem ??.* Let  $\epsilon > 0$ . Let  $\mathbf{A} \in [0, 1]^{N \times N}$  be an adjacency matrix. Let  $0 < d \leq 1$ .

Consider the matrix  $\tilde{\mathbf{A}}$  with entries

$$\tilde{a}_{n,m} = -\log(1 - (1 - d)a_{n,m}).$$

In the following construction, we build IeClam affiliation features  $\mathbf{F}$  and we want

$$1 - \exp(-\mathbf{f}_n^\top \mathbf{L} \mathbf{f}_m) \approx (1 - d)a_{n,m}.$$

Note that  $-\log(1 - (1 - d)a_{n,m})$  is increasing in  $(1 - d)a_{n,m}$ . For  $(1 - d)a_{n,m} = 0$  the value of this function is 0, and for  $(1 - d)a_{n,m} = 1 - d$  it is  $R = -\log(d)$ . Choose  $d = \epsilon/2$ . For this specific choice of  $d$ , if we replace  $d$  by  $\epsilon/2$  in the definition of  $D_\square$  and omit the infimum, we get an upper bound of  $D_\square(\mathbf{P}, \mathbf{A})$ .

Using the overlapping weak regularity lemma, we approximate  $\tilde{\mathbf{A}}$  by an ICG with

$$K = \frac{-9 \log(\epsilon/2)^2}{\epsilon^2}$$

communities,

$$\mathbf{C} = \mathbf{Q} \text{diag}(\mathbf{r}) \mathbf{Q}^\top,$$

such that

$$\|\tilde{\mathbf{A}} - \mathbf{C}\|_\square \leq \epsilon/2.$$

Let  $\mathbf{r}_+ = \text{ReLU}(\mathbf{r})$  and  $\mathbf{r}_- = \text{ReLU}(-\mathbf{r})$ . Denote

$$\mathbf{C}_+ = \mathbf{Q} \text{diag}(\sqrt{\mathbf{r}_+})$$

and

$$\mathbf{C}_- = \mathbf{Q} \text{diag}(\sqrt{\mathbf{r}_-}).$$

Denote the rows of  $\mathbf{C}_+$  by  $\mathbf{t}_n$  and the rows of  $\mathbf{C}_+$  (it should be  $\mathbf{C}_-$ ) by  $\mathbf{s}_n$ , for  $n = 1, \dots, N$ . For each  $n \in [N]$  we concatenate  $(\mathbf{t}_n, \mathbf{s}_n)$  to define the affiliation feature  $\mathbf{f}_n$ , with the first  $K$  coordinates being the inclusive communities, and the last  $K$  coordinates being the exclusive communities. Denote the corresponding IeClam matrix by  $\mathbf{P}$ .

It is easy to see that  $\mathbf{f}_n^\top \mathbf{L} \mathbf{f}_m$  is the  $(n, m)$  entry of  $\mathbf{Q} \text{diag}(\mathbf{r}) \mathbf{Q}^\top$ . This also proves that

$$\begin{aligned} & \| -\log(1 - (1 - \epsilon)\mathbf{A}) + \log(1 - \mathbf{P}) \|_\square \\ &= \|\tilde{\mathbf{A}} - \mathbf{C}\|_\square \leq \epsilon/2. \end{aligned}$$

We can now summarize

$$\begin{aligned} & D_\square(\mathbf{P} \|\mathbf{A}) \leq \\ & \epsilon/2 + \frac{1}{N^2} \sup_{\mathcal{U}, \mathcal{V} \subseteq [N]} \left| \log \left( \prod_{n \in \mathcal{U}} \prod_{m \in \mathcal{V}} \frac{1 - p_{n,m}}{1 - (1 - \epsilon/2)a_{n,m}} \right) \right| \\ &= \epsilon/2 + \frac{1}{N^2} \sup_{\mathcal{U}, \mathcal{V} \subseteq [N]} \left| \sum_{n \in \mathcal{U}} \sum_{m \in \mathcal{V}} \left( -\log(1 - (1 - \epsilon)a_{n,m}) \right. \right. \\ & \quad \left. \left. + \log(1 - p_{n,m}) \right) \right| \\ &= \epsilon/2 + \|\tilde{\mathbf{P}} - \mathbf{C}\|_\square = \epsilon. \end{aligned}$$

□

## A.5 Proof of the Universality of IeClam in the Pairwise Cone of Non-negativity

For this result, we use the standard weak regularity lemma for non-intersecting classes. It is based on the weak regularity lemma from (??), see also Lemma 9.3 and Corollary 9.13 from (??).

**Definition 16.** A block matrix  $\mathbf{B}$  with  $K$  classes is a symmetric matrix  $\mathbf{B} \in [0, \infty)^{N \times N}$  for which there exists a partition of  $[N]$  into  $K$  disjoint sets, called classes,  $\mathcal{C}_1, \dots, \mathcal{C}_K$  (with  $\cup \mathcal{C}_j = [N]$ ), such that for every pair of classes  $i, j \in [K]$ , there is a constant  $c_{i,j} \geq 0$  such that  $b_{n,m} = c_{i,j}$  for any two nodes  $n \in \mathcal{C}_i$  and  $m \in \mathcal{C}_j$ .

**Theorem 17.** Let  $\mathbf{A} \in [0, R]^{N \times N}$  be an adjacency matrix of a graph with  $N$  nodes. Let  $\epsilon > 0$ . Denote  $K = 2^{2\lceil R^2/\epsilon^2 \rceil}$ . Then, there exists a block matrix  $\mathbf{B}$  with  $K$  (disjoint) classes such that

$$\|\mathbf{A} - \mathbf{B}\|_\square \leq \epsilon. \quad (23)$$

We stress that Theorem ?? guarantees non-negative block values of  $\mathbf{B}$ , while in Theorem ??, in general, the matrix  $\mathbf{C}$  may have negative entries.

*Proof of Theorem ??.* We start similarly to the proof of Theorem ?? . Let  $\epsilon > 0$ . Let  $\mathbf{A} \in [0, 1]^{N \times N}$  be an adjacency matrix. Consider the matrix  $\tilde{\mathbf{A}} \in [0, -\log(\epsilon/2)]$  with entries

$$\tilde{a}_{n,m} = -\log(1 - (1 - \epsilon/2)a_{n,m}).$$

In the following, we build IeClam affiliation features  $\mathbf{F}$  such that

$$1 - \exp(-\mathbf{f}_n^\top \mathbf{L} \mathbf{f}_m) \approx (1 - \epsilon/2)a_{n,m}.$$

By the weak regularity lemma (Theorem ??), we approximate there is a non-negative block matrix  $\mathbf{B}$  with  $K = 2^{2\lceil -\log(\epsilon/2)^2/\epsilon^2 \rceil}$  classes  $\mathcal{C}_1, \dots, \mathcal{C}_K$ , such that

$$\|\tilde{\mathbf{A}} - \mathbf{B}\|_\square \leq \epsilon.$$

We now take the affiliation space to have  $C = K^2$  inclusive communities and  $C = K^2$  exclusive communities.

For each  $n \in [N]$ , let  $k_n$  be the class such that  $c \in \mathcal{C}_{k_n}$ . For each pair of classes  $i, j \in [K]$ , let  $c_{i,j} \geq 0$  denote the edge weight between  $\mathcal{C}_i$  and  $\mathcal{C}_j$ .

The feature of each  $n \in [N]$  at the inclusive channel  $c = (K - 1)k_n + k_n$  is  $t_n^c = \sqrt{c_{k_n, k_n}}$ . It is  $t_n^c = \sqrt{c_{k_n, j}}/4$  at inclusive channels  $c = (K - 1)k_n + j$  and  $c = (K - 1)j + k_n$ , and  $s_n^c = -\sqrt{c_{k_n, j}}/4$  at exclusive channels  $c = (K - 1)k_n + j$  and  $c = (K - 1)j + k_n$ . In all other channels  $t_n^c$  and  $s_n^c$  are zero. Note that  $\mathbf{f}_n$  belongs to the cone of pairwise non-negativity  $\mathcal{T} \subset \mathbb{R}^{2K^2}$ .

It is now direct to see that  $\mathbf{f}_n^\top \mathbf{L} \mathbf{f}_m = c_{k_n, k_m}$ . As a result, as in the proof of Theorem ??, we get

$$\begin{aligned} & D_\square(\mathbf{P} \|\mathbf{A}) \leq \\ & \epsilon/2 + \frac{1}{N^2} \sup_{\mathcal{U}, \mathcal{V} \subseteq [N]} \left| \log \left( \prod_{n \in \mathcal{U}} \prod_{m \in \mathcal{V}} \frac{1 - p_{n,m}}{1 - (1 - \epsilon/2)a_{n,m}} \right) \right| \\ &= \epsilon/2 + \frac{1}{N^2} \sup_{\mathcal{U}, \mathcal{V} \subseteq [N]} \left| \sum_{n \in \mathcal{U}} \sum_{m \in \mathcal{V}} \left( -\log(1 - (1 - \epsilon)a_{n,m}) \right. \right. \end{aligned}$$

$$+ \log(1 - p_{n,m})) \Big) \Big| \\ = \epsilon/2 + \|\tilde{\mathbf{P}} - \mathbf{B}\|_{\square} = \epsilon. \quad \square$$

**[Ron: formalize what is universality in log likelihood the following is a lemma which proves universality]**

### A.6 Proof that Convergence in Log Cut Implies Convergence in Log Likelihood

For a binary matrix  $A \in \{0,1\}^{N \times N}$  We denote

$$\sum_{m \sim n} = \sum_{\substack{n \in [N] \\ m \in [N] \\ A_{nm}=1}}$$

and

$$\sum_{m \not\sim n} = \sum_{\substack{n \in [N] \\ m \in [N] \\ A_{nm}=0}}. \quad (24)$$

**Definition 18.** Let  $A \in \{0,1\}^{N \times N}$  and  $P \in [0,1]^{N \times N}$ . The **log likelihood loss** between  $A$  and  $P$  is defined to be:

$$l(P, A) = \sum_{m \sim n} \log(p_{nm}) + \sum_{m \not\sim n} \log(1 - p_{nm}).$$

**[Ron: For this definition, you need to assume  $p_{n,m} > 0$  for every  $n \sim m$ . You can also assume simply that  $p_{n,m} > 0$  for every  $n, m$ , as our methods satisfy this.]**

**Lemma 19.** Let  $\frac{1}{N^2} > \epsilon > 0$ ,  $A \in \{0,1\}^{N \times N}$  and  $P \in [0,1]^{N \times N}$  such that  $D_{\square}(P||A) < \epsilon$ , then

$$|l(P, A)| < 5N^2 E \epsilon$$

where  $E$  is the number of edges in  $A$ .

**[Ron: This does not seem to give universality. Given  $\epsilon$ , it is not true that you can choose a dimension of the code space such that the log likelihood has distance less than  $\epsilon$  from zero, independently of  $N$ . For any  $\epsilon$ , if  $N > \frac{1}{\sqrt{\epsilon}}$  then your theorem does not give anything...]**

*Proof.* We start by proving a number of claims.

**Claim 20.** There exists  $0 < d \leq 1$  such that

$$\epsilon > d + \|\log(1 - p_{nm}) - \log(1 - (1 - d)a_{nm})\|_{\square}$$

*Proof.* If we assume the opposite towards contradiction we get that for every  $d$

$$\epsilon \leq d + \|\log(1 - p_{nm}) - \log(1 - (1 - d)a_{nm})\|_{\square}$$

and therefore  $\epsilon \leq D_{\square}(P||A)$  in contradiction.  $\square$

**Claim 21.** Let  $S$  and  $T$  such that

$$\epsilon > \|S - T\|_{\square}$$

then

$$\forall (n, m) \in [N]^2 : \epsilon > \frac{1}{N^2} |S_{nm} - T_{nm}|$$

*Proof.* For the sets  $\{m\}$  and  $\{n\}$  we get that

$$\epsilon > \|S - T\|_{\square} = \frac{1}{N^2} \sup_{U, V} \left| \sum_{k \in U} \sum_{l \in V} (S_{kl} - T_{kl}) \right| \geq \frac{1}{N^2} \left| \sum_{k \in \{n\}} \sum_{l \in \{m\}} (S_{kl} - T_{kl}) \right| = \frac{1}{N^2} |S_{nm} - T_{nm}|. \quad \square$$

By Claim ?? we deduce that there exists  $d$  such that

$$\epsilon > d \\ \epsilon > \|\log(1 - p_{nm}) - \log(1 - (1 - d)a_{nm})\|_{\square} \quad (25)$$

From Equation ?? and the supremum definition

$$\begin{aligned} \epsilon &> \frac{1}{N^2} \left| \sum_{n \sim m} (\log(1 - p_{nm}) - \log(d)) + \sum_{n \not\sim m} \log(1 - p_{nm}) \right| = \\ &= \frac{1}{N^2} \left| \sum_{n \sim m} (-\log(d) + \log(1 - p_{nm})) - \sum_{n \sim m} \log(p_{nm}) - |l(P, A)| \right| \end{aligned} \quad (26)$$

To handle the  $(-\sum_{n \sim m} \log(p_{nm}))$  term in Equation ?? we prove the following claim.

**Claim 22.**  $|\log(p_{nm})| < 3\epsilon$

*Proof.* We use Claim ?? on the pairs  $(n, m) \in [N]^2$  such that  $a_{nm} = 1$ , so for every such pair it holds that

$$\epsilon > \frac{1}{N^2} \left| \log(1 - p_{nm}) - \log(d) \right| \quad (27)$$

If  $\log(d) < \log(1 - p_{nm})$ , we get that

$$\frac{1 - p_{nm}}{d} < e^{N^2 \epsilon}$$

which implies (recalling that  $\epsilon < \frac{1}{N^2}$ )

$$\begin{aligned} -\log(p_{nm}) &< -\log(1 - de^{N^2 \epsilon}) < -\log(1 - \epsilon e^{N^2 \epsilon}) < \\ &< \epsilon e^{N^2 \epsilon} - \mathcal{O}(\epsilon e^{N^2 \epsilon^2}) < 3\epsilon \end{aligned}$$

If  $\log(d) > \log(1 - p_{nm})$  then

$$\begin{aligned} -\log(1 - p_{nm}) &> -\log(\epsilon) \\ \iff p_{nm} &> 1 - \epsilon \\ \iff -\log(p_{nm}) &< \epsilon - \mathcal{O}(\epsilon^2) < \epsilon \end{aligned} \quad (28)$$

$\square$

We now use Equation ?? and Claim ?? on Equation ?? to bound  $|l(P, A)|$ . By the triangle inequality **[Ron:  $4N^2 \epsilon$  in the left hand side?]**

$$\begin{aligned} N^2 \epsilon &> \left| \sum_{n \sim m} (-\log(d) + \log(1 - p_{nm})) - \sum_{n \sim m} \log(p_{nm}) \right| \\ &= \left| l(P, A) \right|. \end{aligned} \quad (29)$$

Finally

$$|l(P, A)| < (N^2 + N^2 E + 3E) \epsilon < 5N^2 E \epsilon. \quad \square$$

## A.7 Proof that a Sigmoid Inner Product Decoder Is Not Universal in Log Likelihood

Consider the bipartite graph  $\mathbf{B} \in \{0,1\}^{2N \times 2N} = (b_{nm})_{n,m \in [2N]}$  with  $N$  nodes in each part and denote the parts by  $\mathcal{C}_i$  where  $i \in \{1, 2\}$ . Arrange the nodes such that  $\mathcal{C}_1 = [N]$  and  $\mathcal{C}_2 = [N] + N$ .

Consider Definition (??) as the definition of the log likelihood. Let  $\sigma : \mathbb{R} \rightarrow [0, 1]$  be an ascending function. Also, let  $K$ , and  $\Pi : \mathbb{R}^K \times \mathbb{R}^K \rightarrow [0, 1]$  such that  $\Pi(\mathbf{f}_n, \mathbf{f}_m) = \sigma(\mathbf{f}_n^\top \mathbf{f}_m)$ . Let  $\mathbf{P}$  such that  $p_{nm} = Pr(n \sim m \mid \mathbf{f}_n, \mathbf{f}_m) = \Pi(\mathbf{f}_n, \mathbf{f}_m)$  be a random graph reconstruction from the affiliation features  $\mathbf{F}$ . Note that BigClam is a special case of this construction.

**Claim 23.** *Under the above construction,*

$$|ll(P, B)| \geq -\log\left(\frac{1}{2}\right)$$

*As a result, any autoencoder that uses a monotonous ascending function of the inner product between affiliation features for decoding is not universal.*

*Proof.* For  $n \in [N]$  we denote  $\mathbf{q}_n = \mathbf{f}_n$  and  $\mathbf{y}_n = \mathbf{f}_{n+N}$ . Let  $a > 0$  such that  $|ll(P, B)| \leq a$ . Therefore

$$\begin{aligned} a > & -2 \sum_{n=1}^N \sum_{m=1}^N \log(\sigma(\mathbf{q}_n^\top \mathbf{y}_m)) - \sum_{n=1}^N \sum_{m=1}^N \log(1 - \sigma(\mathbf{y}_n^\top \mathbf{y}_m)) \\ & - \sum_{n=1}^N \sum_{m=1}^N \log(1 - \sigma(\mathbf{q}_n^\top \mathbf{q}_m)) \end{aligned} \quad (30)$$

Since all terms are of the same sign it holds that each of them is smaller than  $a$ .

$$-\log(\sigma(\mathbf{q}_n^\top \mathbf{y}_m)) < a \quad (31)$$

and (WLG for  $\mathbf{q}$ )

$$-\log(1 - \sigma(\mathbf{q}_n^\top \mathbf{q}_m)) < a \quad (32)$$

So for every  $n, m \in [N]$

$$\sigma^{-1}(1 - e^{-a}) > \max(\|\mathbf{q}_n\|^2, \|\mathbf{y}_m\|^2) > \mathbf{q}_n^\top \mathbf{y}_m > \sigma^{-1}(e^{-a})$$

$\sigma^{-1} : [0, 1] \rightarrow \mathbb{R}$  is also ascending so we must have  $1 - e^{-a} > e^{-a} \iff a > \log(\frac{1}{2})$  which means that for  $a$  to be larger than  $|ll(P, B)|$ ,  $a$  has to be larger than  $\log(\frac{1}{2})$  and so

$$|ll(P, B)| \geq \log\left(\frac{1}{2}\right)$$

□

## B Extended Related Work

### B.1 Message Passing Algorithms and Networks

The message passing algorithm is a general architecture for processing graph-signals. An MPNN operates on the graph data by aggregating the features in the neighborhood of each node, allowing for information to travel along the edges. The first example of this scheme (Message Passing) was originally suggested by Pearl et al (?), and was combined with a neural network in (?), and later generalized in (?). Most

graph neural networks applied in practice are specific instances of MPNNs (??).

In MPNNs, information is exchanged between nodes along the graph's edges. Each node combines the incoming messages from its neighbors using an *aggregation scheme*, with common methods being summing, averaging, or taking the coordinate-wise maximum of the messages. Let  $T \in \mathbb{N}$  represent the number of layers, and define two sequences of positive integers  $(c_t)_{t=0}^T$  and  $(d_t)_{t=0}^T$  representing the feature dimensions in the hidden layers  $\{\mathcal{G}_t\}_{t=0}^T = \{\mathbb{R}^{c_t}\}_{t=0}^T$  and  $\{\mathcal{S}_t\}_{t=0}^T = \{\mathbb{R}^{d_t}\}_{t=0}^T$ . Define the message functions as  $M^t : \mathcal{S}_t \times \mathcal{S}_t \rightarrow \mathcal{G}_t$  and update functions as  $U^t : \mathcal{G}_t \times \mathcal{S}_t \rightarrow \mathcal{S}_{t+1}$ . The features  $\mathbf{f}_n^{t+1} \in \mathcal{S}_{t+1}$  at layer  $t + 1$  of the nodes  $n \in [N]$  are computed from the features  $\mathbf{f}_m^t \in \mathcal{S}_t$  by

$$\begin{aligned} \mathbf{m}_n^t &= \sum_{k \in \mathcal{N}(n)} M^t(\mathbf{f}_n^t, \mathbf{f}_k^t) \\ \mathbf{f}_n^{t+1} &= U^t(\mathbf{m}_n^t, \mathbf{f}_n^t). \end{aligned}$$

Here,  $M^t$ ,  $U^t$  and  $m^t$  are called the message function, update function and mail at time  $t$  respectively. The summation over the messages can also be replaced for any node by any function  $Agg_n^t : \prod_{k \in \mathcal{N}(n)} \mathcal{G}_t \rightarrow \mathcal{G}_t$  which is permutation invariant.

At step  $T$  a readout space can be defined  $\mathcal{R}_T = \mathbb{R}^{b_T}$ , with a permutation invariant readout function  $R^T$ , e.g., summing, averaging, or taking the max of all of the nodes of the graph. This produces a vector representation of the whole graph.

### B.2 Deep Generative Models

Generative models in machine learning assume that training data is generated by some underlying probability distribution. One goal in this context is to approximate this distribution, or build a model that approximates a random sampler of data points from this distribution. Hence, generative models can be used to generate synthetic data, mimicking training data by sampling from the distribution (??), or to infer the probability of unseen data by substituting it into the probability function. The latter can be useful in tasks like anomaly detection (?) in which the model can assess whether a sample is probable under the learned model. Two examples of generative models are Generative Adversarial Networks (GANs) (??) and Variational Autoencoders (VAEs) (?), which are used both for inference and generation.

VAE models consist of an encoder and a decoder. The encoder maps data from a high-dimensional *data space* to a lower-dimensional, simpler, *code space*. The decoder reverses this process, transforming data from the code space back into the data space. The code space serves as a bottleneck, capturing the essential features of the training data, which is often high-dimensional (e.g., images, social network graphs) and thus more complex than the code space. If the model trains by encoding followed by decoding, then minimizing the difference between the input and its decoded version, it is called an *autoencoder*.

In a VAE, training involves encoding each data point to a known distribution (typically Gaussian), sampling from this distribution, decoding it, and then minimizing the difference

between the distribution of the original data and the decoded data. For a survey on VAEs, see (?).

### B.3 Graph Generative Models

Graph generative models learn a probability distributions of graphs. Such models allow for various tasks where the goal is not only to analyze existing graphs but also to predict or simulate new graph data.

Some classical generative models are pre-defined probabilistic models, e.g., the Erdős–Rényi model, Preferential attachment, Watts–Strogatz model, and more. See (?) for a review. Other graph generative models are learned from data, e.g., (????).

Applications of generative graph models include social network analysis (???), anomaly detection ??, graph synthesis (?), data augmentation (?), and protein interaction modeling (e.g. for drug manufacturing) (?), to name a few. For a review see (?).

**Deep Graph Autoencoders.** In graph deep learning-based autoencoders, one estimates the data distribution by learning to embed the nodes to a code space, in which the data distribution is defined to be some standard distribution, e.g., Gaussian, in such a way that the encoded nodes can be reconstructed back to the graph with small error. Graph VAEs (????) embed the data into the code space by minimizing the evidence lower bound loss comprised of the decoding loss and the KL divergence between the encoded distribution and a Gaussian prior.

**GAN-based Graph generative models.** In (?), a GAN method for graphs generates a neighborhood for each of the nodes and the discriminator gives a probability score for each edge. This method also formulate a graph version of softmax, and offer a random walk based generating strategy. Another GAN model that is used for anomaly detection is GAAN (?). In GAAN, the ground truth and generated node attributes are encoded into a latent space from which the adjacency between any two nodes is decoded using a sigmoid of the inner product between their latent features.

**Normalizing Flows-based Graph Models.** Normalizing flows models (?) also have adaptations for graph data. For example, the work of (??) offers a version of coupling blocks that use message passing neural networks. See more details on normalizing flows in Section ??.

### B.4 Stochastic block models

A stochastic block model (SBM) is a generative model for random graphs. A basic stochastic block model is defined by specifying a number of *classes*  $K$ , the probability  $p_k$  of a random node being in block  $k$ , for  $k \in [K]$ , where  $\sum_k p_k = 1$ , and an array of values  $\mathbf{C} = \{c_{k,l}\}_{k,l=1}^J \in [0, 1]^{J \times K}$  indicating edge probabilities between classes. Each node of a randomly generated graph of  $N$  nodes is independently chosen to belong to one of the classes at random, with probabilities  $\{p_k\}_{k \in [K]}$ . Then, the edges of the graph are chosen independently at random according to the following rule. For each  $n \in [N]$ , denote by  $k_n \in [K]$  the class of  $n$ . Each dyad  $(n, m) \in [N]^2$  is chosen to be an edge in probability  $c_{k_n, k_m}$ .

Namely, the entries  $a_{n,m}$  of the adjacency matrix of the random graph are independent random Bernoulli variables. See (?) for a review on SBMs.

For each node  $n$ , denote by  $\mathbf{f}_n \in \{0, 1\}^K$  the vector such that  $f_n^c = 1$  if and only if  $k_n = c$ . Hence, in a basic SBM, the presence of an edge between nodes  $n$  and  $m$  follows a Bernoulli distribution with parameter  $P(n \sim m | \mathbf{f}_n, \mathbf{f}_m, \mathbf{C}) = \mathbf{f}_n^\top \mathbf{C} \mathbf{f}_m$ , where the adjacency matrix is  $\mathbf{P} = \mathbf{F} \mathbf{C} \mathbf{F}^\top$ , where  $\mathbf{F} \in \mathbb{R}^{N \times K}$  is the matrix where each row  $n$  has the feature  $\mathbf{f}_n$  (?). This model can be extended to intersecting classes, where now  $\mathbf{f}_n$  can have more than one nonzero entry (???).

In a *Bernouli-Poisson SBM* (???), the probability for a non-edge is modeled by a Poisson distribution. The idea is that the more classes  $n$  and  $m$  share, the higher the probability that there is an edge between them. Hence,

$$P(n \sim m | \mathbf{f}_n, \mathbf{f}_m, \mathbf{C}) = 1 - e^{-\mathbf{f}_n^\top \mathbf{C} \mathbf{f}_m}, \quad (33)$$

with the expected number of edges being  $\mathbf{f}_n^\top \mathbf{C} \mathbf{f}_m$ .

In both the Bernouli and Bernouli-Poisson models, the probabilistic model of the entire graph is given by a product of the probabilities of all of the events

$$P(E | \mathbf{F}, \mathbf{C}) = \sqrt{\prod_{n \in [N]} \left( \prod_{m \in \mathcal{N}(n)} P(n \sim m) \prod_{m \notin \mathcal{N}(n)} P(n \not\sim m) \right)}.$$

Here, the square root is taken since we assume that the graph is undirected so the product goes over all of the edges twice (??).

When fitting an SBM to a graph, both the class affiliations of nodes and the block structure  $\mathbf{C}$  are learned (???).

### B.5 Community Affiliation Models

Community detection is a fundamental task in network analysis, aiming to identify groups of nodes that are more densely connected internally than with the rest of the network. This process is useful for understanding the structure and function of complex networks, such as social, biological, and information networks (?).

Although there exist models in which each node belongs to only one community as in traditional SBM models (?), and some relatively new deep learning models such as (?), it was shown that real-world networks often exhibit overlapping communities (??), where nodes belong to multiple groups and the probability of connectivity increases the more communities two nodes share. This indeed makes intuitive sense when looking at, e.g., social networks, where the more common interests and social circles people share the more they are likely to connect.

An example of a community affiliation model that can generate new graphs is AGM (?), which classifies all of the nodes in a graph into several communities, where each community  $k$  has a probability  $p_k$  for two member nodes to connect. If we denote by  $C_{n,m}$  the set of communities two nodes  $n, m \in [N]$  share, then the probability of an edge between  $n$  and  $m$  is

$$p(n \sim m) = 1 - \prod_{k \in C_{n,m}} (1 - p_k).$$



To sample a new graph from a trained model, nodes are sampled and assigned communities based on the relative sizes of communities in the training graph, and edges are connected based on their mutual community memberships.

Community affiliations can also be continuous, where nodes have varying degrees of membership in multiple communities. Community Affiliation models with continuous communities include Mixed Membership SBM (MMSBM) (?) which is an SBM which allows nodes to have mixed memberships in multiple communities, and BigClam (?), which is a Bernouli Poisson model model that scales efficiently for sparse graphs.

It is also worth mentioning the NOCD model presented in (?) which uses the Bernouli Poisson loss, but embeds the nodes of the graph into the code space using a learned GNN. Another related paper is (?), which models the features and communities separately, learning latent features from which it estimates the community affiliation.

While community affiliations are typically non-negative, there are models where affiliations can be negative. An example is the Signed Stochastic Block Model (SSBM) (?).

## B.6 Anomaly Detection

Anomaly detection in graphs aims to identify nodes or sub-graphs that deviate significantly from the typical patterns within the graph. This is useful in various applications such as network security, fraud detection, and social network analysis. In the unsupervised formulation of the problem, there is no labeled data in the training process. We highlight five works in this direction. GAAN (?) and AEGIS (?) use a generative adversarial approach, training a discriminator to distinguish real and fake nodes. Dominant (?) and AnomalyDAE (?) identify anomalies via reconstruction errors of a graph autoencoder.

Another work is (?), which considers a semi-supervised setting. Here, there is a relatively small number of available labeled normal nodes during training (nodes that are known to not be anomalies), and the goal is to predict the anomalies in the unknown nodes.

## B.7 Link Prediction

Link prediction in graphs aims to predict missing or future connections between nodes based on the existing structure and node features. This task is critical in various domains such as social networks, biological networks, and recommendation systems. In the supervised setting, a subset of dyads is withheld as a test set, with their labels (edges or non-edges) omitted. The goal is to predict the connectivity of these omitted dyads using the remaining data, which may also include node features.

We highlight five notable works in this area. The Adamic-Adar index (?) henceforth referred to as AA, is a second-order heuristic that assumes shared neighbors with lower degrees contribute more to the likelihood of a link, while high-degree neighbors are less significant. VGAE (?) is a variational graph autoencoder, discussed in Section ?? . LINKX (?) decouples the feature and structure representations in a graph and uses a simple framework to process this information separately. GAT (?) applies an attention mechanism in

the message passing process, allowing the model to weight neighboring nodes based on their importance. Another recent work, DisenLink (?), learns disentangled representations of nodes by capturing multiple latent factors, allowing it to model various aspects of node connectivity for link prediction. All models mentioned utilize node features except for AA.

## B.8 Normalizing Flows

Normalizing flows are deep learning algorithms that estimate probability distributions, and allow an efficient sampling from this distribution. They do so by constructing an invertible coordinate transformation between the unknown target probability space and a standard probability space with a well known distribution (e.g. Gaussian). This transformation is modeled as a deep neural network, composed of a series of basic transformations called *flows*.

**Notations.** Let  $\mathcal{F}$  represent the space of the target data, with an unknown probability density function  $p_{\mathcal{F}} : \mathcal{F} \rightarrow [0, \infty)$ . We denote the elements of  $\mathcal{F}$  by  $\mathbf{f}$ .

Let  $\mathcal{Z}$  represent a latent space with a known probability density function  $p_{\mathcal{Z}} : \mathcal{Z} \rightarrow [0, \infty)$ . We denote the elements of  $\mathcal{Z}$  by  $\mathbf{z}$ . The density function  $p_{\mathcal{Z}}$  is often chosen to be a standard isotropic Gaussian with zero mean and identity covariance, denoted by  $\mathcal{N}(0, \mathbf{I})$ .

**Goal.** The goal in normalizing flows is to learn an invertible transformation  $T_{\theta} : \mathcal{F} \rightarrow \mathcal{Z}$  (parameterized by  $\theta$ ) which maps the target space  $\mathcal{F}$  to the latent space  $\mathcal{Z}$ , and preserves probabilities. Namely,  $T_{\theta}$  should satisfy: for every measurable subset  $F \subset \mathcal{F}$  we have

$$\int_F T_{\theta}(\mathbf{f}) p_{\mathcal{F}}(\mathbf{f}) d\mathbf{f} = \int_{T_{\theta}(F)} p_{\mathcal{Z}}(\mathbf{z}) d\mathbf{z}.$$

Here  $T_{\theta}(F) = \{z \in \mathcal{Z} \mid \exists \mathbf{f} \in F \text{ such that } T_{\theta}(\mathbf{f}) = \mathbf{z}\}$ . Such a transformation can be seen as a change of variable.

**Density Transformation.** Given an invertible transformation  $T_{\theta}$ , the target density  $p_{\mathcal{F}}(\mathbf{f})$  can be expressed in terms of the latent density  $p_{\mathcal{Z}}(\mathbf{z})$ . By upholding the constraint that either probability density function has integral 1 over their respective spaces, one can deduce the change of variable formula

$$p_{\mathcal{F}}(\mathbf{f}) = p_{\mathcal{Z}}(T_{\theta}(\mathbf{f})) \left| \det \left( \frac{\partial T_{\theta}(\mathbf{f})}{\partial \mathbf{f}} \right) \right|.$$

Here,  $\frac{\partial T_{\theta}(\mathbf{f})}{\partial \mathbf{f}}$  denotes the Jacobian matrix of the transformation  $T_{\theta}$  with respect to  $\mathbf{f}$  and  $\det(\cdot)$  denotes its determinant.

**Sequential Composition of Flows.** In practice, the transformation  $T_{\theta}$  is modeled as a composition of  $L \in \mathbb{N}$  simpler invertible transformations  $\{T_{\theta^i}\}_{i=1}^L$  between the consecutive spaces  $\{\mathcal{Z}^i\}$  where  $\mathcal{Z}^L = \mathcal{F}$  and  $\mathcal{Z}^1 = \mathcal{Z}$  in the previous notations. The invertible mappings  $T_{\theta^i} : \mathcal{Z}^i \rightarrow \mathcal{Z}^{i-1}$  are called flows, and we have

$$T_{\theta} = T_{\theta^L} \circ T_{\theta^{L-1}} \circ \dots \circ T_{\theta^1}$$

where  $\circ$  denotes function composition. The overall density function for  $\mathbf{f}$  then becomes

$$p_{\mathcal{F}}(\mathbf{f}) = p_{\mathcal{Z}}(T_{\theta}(\mathbf{f})) \prod_{i=1}^L \left| \det \left( \frac{\partial T_{\theta^i}(\mathbf{f}_i)}{\partial \mathbf{f}_i} \right) \right|.$$

Here,  $\mathbf{f}_i$  is the intermediate representation after applying the first  $i - 1$  flows. The algorithm is optimized by maximum log likelihood, namely, by maximizing

$$\begin{aligned} & \log(p_{\mathcal{F}}(\mathbf{f})) \\ &= \log(p_{\mathcal{Z}}(T_{\theta}(\mathbf{f}))) + \sum_{i=1}^L \log \left( \left| \det \left( \frac{\partial T_{\theta}^i(\mathbf{f}_i)}{\partial \mathbf{f}_i} \right) \right| \right), \end{aligned}$$

using gradient descent.

Since  $T_{\theta}$  and every  $T_{\theta}^i$  are invertible, information can also flow in the opposite direction in order to sample data. First, a point  $\mathbf{z}$  is sampled  $\mathbf{z} \sim p_{\mathcal{Z}}$  and the inverse transformation  $T_{\theta}^{-1}$  is applied to map the sample  $\mathbf{z}$  into the data space  $\mathcal{F}$ . Since  $T$  is composed of a series of flows, the generated point in the data space is

$$\mathbf{f} = T_{\theta}^{-1}(\mathbf{z}) = T_{\theta^1}^{-1} \circ T_{\theta^2}^{-1} \circ \dots \circ T_{\theta^L}^{-1}(\mathbf{z}).$$

**RealNVP.** One popular method of implementing normalizing flow is the *Real Valued Non-Volume Preserving (RealNVP)*. In this model, the flows  $T_{\theta^i}$  are modeled as mappings called *coupling blocks*. In a coupling block, the input vector  $\mathbf{f}^i$  (or  $\mathbf{z}^i$ , in the generative direction) is split into two vectors:  $\mathbf{f}_A^i$  and  $\mathbf{f}_B^i$ , each the same dimension. Namely,  $\mathbf{f}^i = (\mathbf{f}_A^i, \mathbf{f}_B^i)$ . The flow  $T_{\theta^i}$  is then defined to be

$$\mathbf{f}_A^{i-1} = \mathbf{f}_A^i \quad (34)$$

$$\mathbf{f}_B^{i-1} = \mathbf{f}_B^i \odot \mathbf{s}_{\theta^i}(\mathbf{f}_A^i) + \mathbf{t}_{\theta^i}(\mathbf{f}_A^i) \quad (35)$$

Where  $(\mathbf{s}_{\theta^i}(\cdot), \mathbf{t}_{\theta^i}(\cdot))$  are Multi Layer Perceptrons (MLPs) parameterized by  $\theta^i$ , and  $\odot$  is elementwise product. In addition, the elements of  $\mathbf{f}^i$  are permuted at every  $i$  before applying the coupling block, using predefined permutations, that are parts of the hyperparameters of the model. It is easy to see that the Jacobian  $\left( \frac{\partial T_{\theta}^i(\mathbf{f}_i)}{\partial \mathbf{f}_i} \right)$  is a diagonal matrix with 1 for the first  $\dim(\mathbf{f}_A^i)$  elements, and  $\mathbf{s}_{\theta^i}(\mathbf{f}_A^i)$  for the last  $\dim(\mathbf{f}_B^i)$  elements. This makes the log of the determinant of the Jacobian

$$\log \left( \left| \det \left( \frac{\partial T_{\theta}^i(\mathbf{f}_i)}{\partial \mathbf{f}_i} \right) \right| \right) = \sum_{j=0}^{\dim(\mathbf{f}_B^i)} \log(\mathbf{s}_{\theta^i}(\mathbf{f}_A^i)_j). \quad (36)$$

Hence, this determinant is easily computed in practice.

For generation, the inverse transformation can be calculated easily as

$$\begin{aligned} \mathbf{z}_A^{i+1} &= \mathbf{z}_A^i \\ \mathbf{z}_B^{i+1} &= \frac{\mathbf{z}_B^i - \mathbf{t}(\mathbf{z}_A^i)}{\mathbf{s}(\mathbf{z}_A^i)} \end{aligned}$$

where the division is elementwise. Here, the log determinant of the Jacobian can be calculated similarly to (??), applied to  $1/\mathbf{s}(\mathbf{z}_A^i)$ .

## B.9 The Lorentz Inner Product

The Lorentz inner product is a bilinear form used in the context of special relativity to describe the spacetime structure. In a four-dimensional spacetime, the Lorentz inner product between two vectors  $\mathbf{v}$  and  $\mathbf{w}$  is given by:

$$\langle \mathbf{v}, \mathbf{w} \rangle = v_0 w_0 - v_1 w_1 - v_2 w_2 - v_3 w_3$$

Here,  $v_0$  and  $w_0$  represent the "time" components, while  $v_1, v_2, v_3$  and  $w_1, w_2, w_3$  represent the "spatial" components. The negative sign in front of the time component is what distinguishes the Lorentz inner product from the standard Euclidean inner product, making it suitable for modeling the geometry of spacetime where time and space are treated differently. Note that due to the subtraction, the Lorentz inner product is in fact not an inner product as it is not positive definite. In the context of special relativity, the points for which the Lorentz product remain positive define the so called "light cone" structure of spacetime, separating events into those that are causally connected and those that are not.

The Lorentz inner product is a specific example of a broader class of inner products known as **pseudo-Euclidean inner products**. In a pseudo-Euclidean space, the inner product can have a mixture of positive and negative signs, leading to different geometric properties. These spaces generalize the concept of Euclidean space by allowing for non-positive definite metrics.

## C PClam and PieClam as Graphons

A graphon is a model which can be seen as a graph generative model that extends SBMs. A graphon (??) can be seen as a weighted graph with a "continuous" node set  $[0, 1]$ .

**Definition 24.** The space of graphons  $\mathcal{W}$  is defined to be the set of all measurable function  $W : [0, 1]^2 \rightarrow [0, 1]$  which are symmetric, namely  $W(x, y) = W(y, x)$ .

The edge weight  $W(x, y)$  of a graphon  $W \in \mathcal{W}$  can be seen as the probability of having an edge between node  $x$  and node  $y$ . Given a graphon  $W$ , a random graph is generated by sampling independent uniform nodes  $\{X_n\}$  from the graphon domain  $[0, 1]$ , and connecting each pair  $X_n, X_m$  in probability  $W(X_n, X_m)$  to obtain the edges of the graph.

We next show that Clam models with prior are special cases of graphon models. Note that the prior  $p$  defines a standard atomless probability spaces over the community affiliation space  $\mathbb{R}^K$ . Since all standard atomless probability spaces are equivalent, there is a probability preserving a.e. bijection  $\xi_p : [0, 1] \rightarrow \mathbb{R}^K$  that maps the prior probability to the uniform probability over  $[0, 1]$ . Now, the PieClam model for generating a graph of  $N$  nodes can be written as follows.

- Sample  $N$  points  $\{X_n\}_{n=1}^N \subset [0, 1]$  uniformly. Observe that  $\{\xi_p(X_n)\}_{n=1}^N \subset \mathbb{R}^K$  are independent samples via the probability density  $p$ .
- Connect the points according to the BigClam of IeClam models  $P(n \sim m | \xi_p(X_n), \xi_p(X_m))$  (??) or (??).

This shows that PClam and PieClam coincide with the generative graphon model  $W(x, y) = P(n \sim m | \xi_p(X), \xi_p(Y))$  where  $P$  is defined either by (??) or by (??).

## D Extended Details on Experiments

All of the experiments were run on Nvidia GeForce RTX 4090 and Nvidia L40 GPUs. Our code can be found in:

<https://anonymous.4open.science/r/PieClam-49C4>



| Method         | Reddit               | Elliptic                              | Photo                |
|----------------|----------------------|---------------------------------------|----------------------|
| (S)- IeClam    | <b>0.639</b>         | 0.440                                 | <b>0.592</b>         |
| (S) - PieClam  | $0.6320 \pm 0.0054$  | $0.4354 \pm 0.0005$                   | $*0.5727 \pm 0.0151$ |
| (P) - PieClam  | $0.5089 \pm 0.0219$  | <b><math>0.6201 \pm 0.0176</math></b> | $0.4252 \pm 0.0074$  |
| (PS) - PieClam | $*0.6329 \pm 0.0052$ | <u><math>0.5294 \pm 0.0089</math></u> | $0.5289 \pm 0.0127$  |
| (S) - BigClam  | <u>0.637</u>         | 0.434                                 | <u>0.581</u>         |
| DOMINANT       | 0.511                | 0.296                                 | 0.514                |
| AnomalyDAE     | 0.509                | *0.496                                | 0.507                |
| OCGNN          | 0.525                | 0.258                                 | 0.531                |
| AEGIS          | 0.535                | 0.455                                 | 0.552                |
| GAAN           | 0.522                | 0.259                                 | 0.430                |
| TAM            | 0.606                | 0.404                                 | 0.568                |

Table 3: Comparison of Clam anomaly detectors with competing methods. First place in **boldface**, second with underline, third with \*star. We observe that our methods are first place on all datasets. Moreover, S- IeClam, PS-PieClam and S BigClam each beats the competing methods in two out of the three datasets . The accuracy metric is areas under curve (AUC).

| Method                   | Reddit                                | Elliptic                              | Photo                                 |
|--------------------------|---------------------------------------|---------------------------------------|---------------------------------------|
| (S) - PieClam            | $0.6320 \pm 0.0054$                   | $0.4354 \pm 0.0005$                   | <b><math>0.5727 \pm 0.0151</math></b> |
| (P) - PieClam            | $0.5089 \pm 0.0219$                   | <b><math>0.6201 \pm 0.0176</math></b> | $0.4252 \pm 0.0074$                   |
| (PS) - PieClam           | $*0.6329 \pm 0.0052$                  | <u><math>0.5294 \pm 0.0089</math></u> | $0.5289 \pm 0.0127$                   |
| (S) - PieClam (No Attr)  | <u><math>0.6346 \pm 0.0015</math></u> | $0.4349 \pm 0.0009$                   | $*0.5696 \pm 0.0065$                  |
| (P) - PieClam (No Attr)  | <u><math>0.4530 \pm 0.0142</math></u> | $*0.4826 \pm 0.0134$                  | $0.4977 \pm 0.0747$                   |
| (PS) - PieClam (No Attr) | <b><math>0.6351 \pm 0.0015</math></b> | $0.4349 \pm 0.0009$                   | <u><math>0.5697 \pm 0.0065</math></u> |

Table 4: Comparison of attributed and unattributed PieClam anomaly detection.

### D.1 Additional Architecture Details in PieClam and PClam

**Added Affiliation Noise.** When training the prior, overfitting may cause the probability to spike around certain areas in the affiliation space, e.g., around the affiliation features of the nodes. To avoid this issue, we add gaussian noise to the affiliation vector as a regularization at each step of training the normalizing flow prior model. The normalizing flow model then transforms the same point to a slightly different location in the code space. The noise optimization therefore provides a resolution to the prior, smoothing the distribution in the affiliation space. Noise addition is the primary regularization method we have used when training the prior in all of our experiments.

**Densification.** For very sparse datasets, Clam models may not behave well. In such cases, the community structure is sometimes unstable, where the same node can find itself in different communities based on slightly different initial conditions. In order to strengthen the connections within communities, we apply a two-hop densification scheme on very sparse graphs. Namely, we connect two disjoint nodes  $n, m$  with an edge if there is a third node  $k$  for which  $n \sim k$  and  $m \sim k$ . We find that this scheme improves anomaly detection on Elliptic, Photo and Reddit datasets. Densification may strengthen the community structure in some datasets, but can also destroy it in others. In the experiments on synthetic datasets we did not use densification, as these graphs are relatively dense. *after a closer examination, the elliptic dataset is not improved by densification. this is probably because of the heterophilic properties of the dataset. Instead*

*of going into detail on heterophilic properties, i suggest we look huristically on the datasets and say something like "reddit and photo are social networks so we expect them to have an inclusive structure whereas elliptic for which the nodes represent licit and illicit bitcoin transactions and edges represent flow of money, we would expect illicit transactions to cover their tracks and sift the money into licit transactions therefore having more of an exclusive structure". or we can leave the results as are....*

### D.2 Anomaly detection

**Metric.** In order to measure the accuracy of the anomaly detection classification we use the area under the Receiver Operating Characteristic (ROC) curve (?). The curve is a plot of the true positive rate (TPR) against the false positive rate (FPR) for the range of the threshold values between the value that classifies all samples as true and the value that classifies all as false. The area under the curve signifies the general tendency of the curve toward the point (0,1) for which FPR= 0 and TPR= 1.

**Additional experiment.** We add additional experiments on anomaly detection using PieClam, where we averaged the accuracy over 10 rounds and included error bars.

In this setup we compose three iteration steps of the form  $\mathbf{F}-500 \rightarrow p-1300$  with initial learning rates of  $3e-6$  for the features and  $2e-6$  for the prior, and initial noise amplitude of 0.05. The learning rates and noise amplitude were halved after every  $\mathbf{F} \rightarrow p$  iteration. The results are in Table ??.

Furthermore, in this section we also run our anomaly detection methods without using the node features, namely,

| Method    | Squirrel         | Photo            | Texas            | John’s Hopkins 55 |
|-----------|------------------|------------------|------------------|-------------------|
| PieClam   | <b>98.8 ±0.0</b> | <b>98.4 ±0.0</b> | <b>86.4 ±0.4</b> | 95.6 ±0.0*        |
| BigClam   | 98.5 ± 0.0       | 97.4 ± 0.0*      | 78.8 ± 0.5*      | 94.9 ± 0.0        |
| VGAE      | 98.2 ±0.1        | 94.9 ±0.8        | 68.6 ±4.2        | 92.8 ±0.2         |
| GAT       | 98.0 ±0.0        | 97.3 ±0.3        | 68.5 ±5.4        | 94.3 ±0.5         |
| LINKX     | 98.1 ±0.3        | 97.0 ±0.2        | 75.8 ±4.7        | 93.4 ±0.3         |
| AA        | 97.1 ±0.4        | 97.4 ±0.4        | 53.1 ±6.2        | <u>96.1 ±0.5</u>  |
| DisenLink | 98.3 ±0.1*       | <u>97.9 ±0.1</u> | 80.7 ±4.0        | <b>97.5 ±0.1</b>  |

Table 5: Comparison of Clam link predictors with competing models. First place in **boldface**, second with underline, third with \*star

only using the graph structure. We find that we comparable results on Photo and Reddit with or without node features, but that the detection on Elliptic decreases significantly. Still, anomaly detection on Elliptic without node features is competitive with state of the art models that do use node features. The results are in Table ??.

We proceed to compare the optimization that includes the node features to the optimization that didn’t. The comparison is presented in

**T-Test Comparison Summary.** We conducted t-tests to determine if node features/attributes (“With Features”) significantly improve performance compared to not using attributes (“Without Features”), across the Reddit, Elliptic, and Photo datasets for each method (S, P, PS). Even though S method is not affected directly by the affiliation vectors, the latter can affect the convergence of the prior in the affiliation space and have an indirect effect.

- **Reddit Dataset:** Attributes did not significantly improve performance for the “Star” (S) and “Prior Star” (PS) methods. However, “Prior” (P) did show a significant improvement with attributes (p-value=0.0002).
- **Elliptic Dataset:** “Prior” (P) and “Prior Star” (PS) both showed significant improvements with attributes (p-value<0.0001). However, the “Star” (S) method did not significantly benefit from attributes.
- **Photo Dataset:** The “Star” (S) method did not show significant difference (p-value=0.6864). The “Prior Star” (PS) method, however, did show a significant benefit from attributes (p-value<0.0001), and “Prior” (P) had a borderline significant result (p-value=0.0529).

In conclusion, attributes did not make a difference for the “Star” (S) method across all datasets (as would be expected), and for the “Prior Star” (PS) method in the Reddit dataset. However, the “Prior” (P) and “Prior Star” (PS) methods mostly showed improvement in the Elliptic dataset.

### D.3 Link Prediction

**Probabilistic Model with Omitted Dyads.** When doing link prediction (as explained in length in Section ??) we remove a percentage of all dyads from computation, meaning that the omitted dyads are neither edges nor non-edges. Having non-affiliated dyads will manifest in the probabilistic model.

For the revised probabilistic model we define the following sets containing unordered pairs of edges. Remembering

that  $E$  is the set of edges, we define  $\bar{E}$  as the set of non-edges. We also denote the sets of omitted edges and non-edges by  $\check{E}$  and  $\check{\bar{E}}$ . Finally, we denote the set of all of the omitted dyads by  $\check{\mathcal{E}} = \check{E} \cup \check{\bar{E}}$ . The Clam loss neglecting the omitted dyads  $\hat{l}$  will now be

$$2\hat{l}(\mathbf{F}, E, \check{\mathcal{E}}) = \sum_{(n,m) \in E \setminus \check{E}} \log(1 - e^{-\mathbf{f}_n^\top \mathbf{L} \mathbf{f}_m}) - \sum_{(n,m) \in \check{E} \setminus \check{\bar{E}}} \mathbf{f}_n^\top \mathbf{L} \mathbf{f}_m \quad (37)$$

Which is equivalent to:

$$\begin{aligned} & \sum_{(n,m) \in E \setminus \check{E}} \log(e^{\mathbf{f}_n^\top \mathbf{L} \mathbf{f}_m} - 1) - \sum_{(n,m) \in \mathcal{E}} \mathbf{f}_n^\top \mathbf{L} \mathbf{f}_m + \sum_{n \in [N]} \mathbf{f}_n^\top \mathbf{L} \mathbf{f}_m \\ & + \sum_{(n,m) \in \check{\bar{E}}} \mathbf{f}_n^\top \mathbf{L} \mathbf{f}_m. \end{aligned} \quad (38)$$

The first three terms in ?? are the original Clam loss as described in equations ?? and ??, on the retained edge set  $E \setminus \check{E}$ . The second term is a sum of the feature product for all of the omitted dyads. The number of operations is now  $O(\max\{|E|, |\check{E}|, |\check{\bar{E}}|\})$ , and the Clam optimization is modeled as a heterogeneous MPNN. For the prior models, the prior is added to the loss in the same way as described in Equation ??.

**Experimental Setup** In the experiments, we evaluate the model’s ability to classify edges and non-edges by constructing a ROC curve as described in Section ??.

Note that, since the prior is not used in classification, there is no added benefit from utilizing node features when training the prior, as proposed in ?. Therefore, the link prediction algorithm relies solely on the graph’s topological structure and the prior functions only as regularization.

We use the scheme from (?), where 10% of the edges are omitted as a test set. For each omitted edge, five times that number of non-edges are also omitted. Additionally, for each run of the optimization another 5% of the remaining training edges are picked at random and set aside as a validation set, which is used to optimize hyperparameters for each dataset separately. Each experiment is repeated 10 times, and we calculate the average and standard deviation to report the results along with confidence bounds.

**Datasets and Baselines.** We conduct these experiments on four datasets: Facebook100’s Johns Hopkins 55 (?), Amazon Photo (?), WebKB’s Texas and Wikipedia’s Squirrel (?). We compare ourselves to the baseline methods AA (?), VGAE (?), GAT (?), LINKX (?), DisenLink (?) and BigClam.

**Hyperparameter Configuration.** As was mentioned in ??, hyperparameters are selected separately for each dataset and model by randomly selecting a validation set outside of the test set. Since all four Clam models are a special case of PieClam, we present only the best performing model and treat it as a hyperparameter. We describe the hyperparameter configuration for each of the models in Table ?. The columns of the table signify the Clam model, AS dimension, number of feature iterations, feature step size, number of prior weight iterations, the prior weight step size, the added noise amplitude, s community regularization and the first function in the alternation. The following hyperparameters are constant across all datasets: feature step size (5e-6),  $l_1$  regularization (1), Noise amplitude (0.1) the number of alternations (7), scheduler step size (3) and the scheduler step size (0.5).

**Results** The results on the datasets and the comparison to the other baseline models is presented in Table ?. Baseline results are taken from Table 2 in (?).

#### D.4 Reconstructing Synthetic Priors

We consider two ground-truth synthetic priors  $p_{big} : \mathbb{R}_+^2 \rightarrow [0, \infty)$  in PClam, and  $p_{ie} : \mathcal{T} \rightarrow [0, \infty)$  in PieClam. We sample  $N = 500$  points from each of the priors, decode the corresponding PClam and PieClam graphs, and sample simple graphs from the random Bernoulli edges as shown in figure ?. Then, given these sampled graphs, we reset their affiliation features to random vectors and fit a BigClam and a PClam to the graph sampled from  $p_{big}$ , and an IeClam and a PieClam to the graph that was sampled from  $p_{ie}$ .

The optimization results are shown in Figures ?? and ?. Both the priored and unpriored models effectively learned the feature reconstructions in the feature spaces  $\mathcal{T}$  and  $\mathbb{R}_+^2$ . Notably, the features in the priored models are more localized, supporting the claim that the prior acts as a regularizer.

The priors themselves are well-approximated, as illustrated in Figure ??, with the quality of approximation improving as more nodes are sampled from the priors.

#### D.5 SBM Reconstruction And Distance Convergence

We extend the experiment in Section ?? by considering another synthetic SBM, which is off-diagonal dominant. As in Section ??, we sample a simple graph with  $N = 210$  nodes from the SBM. The non zero probability blocks have a probability of 0.5. We estimate the log cut distance between the Clam models (BigClam and IeClam) and the SBM during training. We consider for both methods affiliation spaces of dimensions 2, 4 and 6. In addition, we calculate the cut distance and l2 errors between the Clam models and the SBM. The results are shown in Figures ??–??.

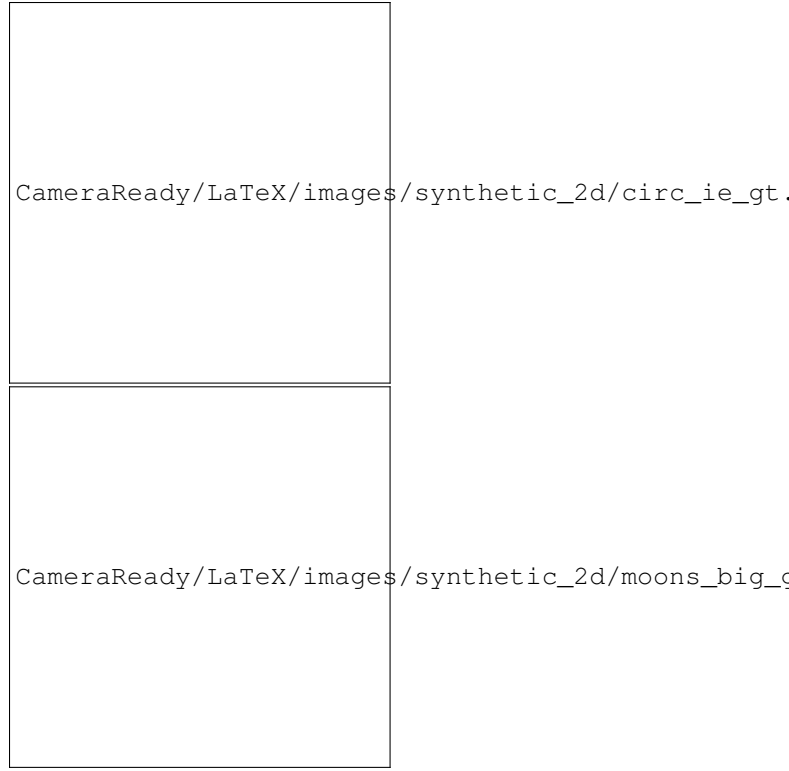


Figure 4: Right to left: ground truth synthetic prior and sampled graph. Top: graph sampled in  $\mathcal{T}$  using PieClam. Bottom: graph sampled in  $\mathbb{R}_+^2$  using Pclam.

#### D.6 Ablation Study

**$l_1$  and s Regularization.** The  $l_1$  regularization is adapted from the BigClam original architecture and shows improvement when optimizing the models. This regularization type adds a constant update in all of the directions, which means that for every node  $n$  and its affiliation feature  $\mathbf{f}_n$ , components that are not updated are moved towards 0 regardless of their magnitude. This constant push towards 0 forces sparsity in the affiliation matrix and also acts as a regularization. The s regularization is an  $l_1$  regularization of the s features and has shown effective for the Squirrel dataset.

**Noise Addition.** Due to the high expressivity of the normalizing flow model, the neural network tends to learn a narrow distribution around the affiliation features. This overfitting tendency cannot be alleviated by simply reducing the network parameters without significantly compromising the expressivity of the prior. Noise addition offers a way to control the prior’s resolution while preserving its ability to learn complex distributions. An example comparing optimization results with and without noise addition is shown in Figure ??.

**Densification.** The densification method described in Section ?? can yield varying results depending on the internal structure of the graph. To illustrate the impact of densification, we conducted 10 anomaly detection experiments, fol-

|          | Model   | Dim. | Feat. Iter. | Prior Iter. | Prior Step | S Reg. | First Optimization |
|----------|---------|------|-------------|-------------|------------|--------|--------------------|
| Squirrel | PieClam | 90   | 700         | 2000        | 5e-6       | 0.1    | Feature            |
| Photo    | PCLam   | 150  | 700         | 1300        | 3e-6       | -      | Feature            |
| Texas    | PieClam | 22   | 2000        | 1500        | 5e-6       | 0.0    | Prior              |
| JH55     | PCLam   | 200  | 750         | 1800        | 3e-6       | -      | Feature            |

Table 6: Link prediction hyperparameters for each of the datasets

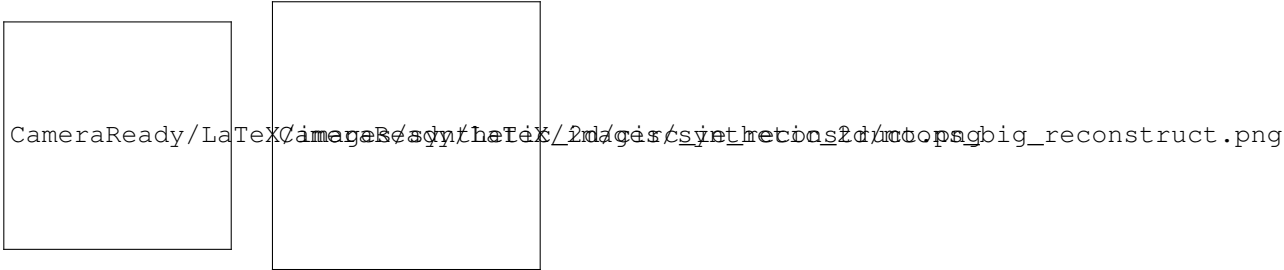


Figure 5: Left to Right: IeClam and BigClam Reconstructions of features sampled from synthetic priors

lowing the setup from Section ??, but without applying densification. The results are shown in Table ??.

| Method                    | Reddit                 | Elliptic               | Photo                  |
|---------------------------|------------------------|------------------------|------------------------|
| (S) - PieClam             | <b>0.6320 ± 0.0054</b> | <b>0.4354 ± 0.0005</b> | <b>0.5727 ± 0.0151</b> |
| (S) - PieClam (No Dens.)  | 0.5480 ± 0.0022        | 0.4009 ± 0.0021        | 0.5293 ± 0.0046        |
| (P) - PieClam             | 0.5089 ± 0.0219        | 0.6201 ± 0.0176        | 0.4252 ± 0.0074        |
| (P) - PieClam (No Dens.)  | 0.5088 ± 0.0383        | <b>0.6378 ± 0.0245</b> | 0.4325 ± 0.0089        |
| (PS) - PieClam            | <b>0.6329 ± 0.0052</b> | 0.5294 ± 0.0089        | <b>0.5289 ± 0.0127</b> |
| (PS) - PieClam (No Dens.) | 0.5466 ± 0.0047        | <b>0.6181 ± 0.0233</b> | 0.4538 ± 0.0030        |

Table 7: Comparison of PieClam on densified and non densified datasets. **Boldface** denotes a significantly better result (difference of over 2 standard deviations), underline denotes an insignificant difference.

As seen in Table ??, densification generally improved results across the datasets, except for the Elliptic dataset using the P and PS methods, where the undensified datasets performed significantly better. For the Reddit and Photo datasets with the P method, the difference was not significant.

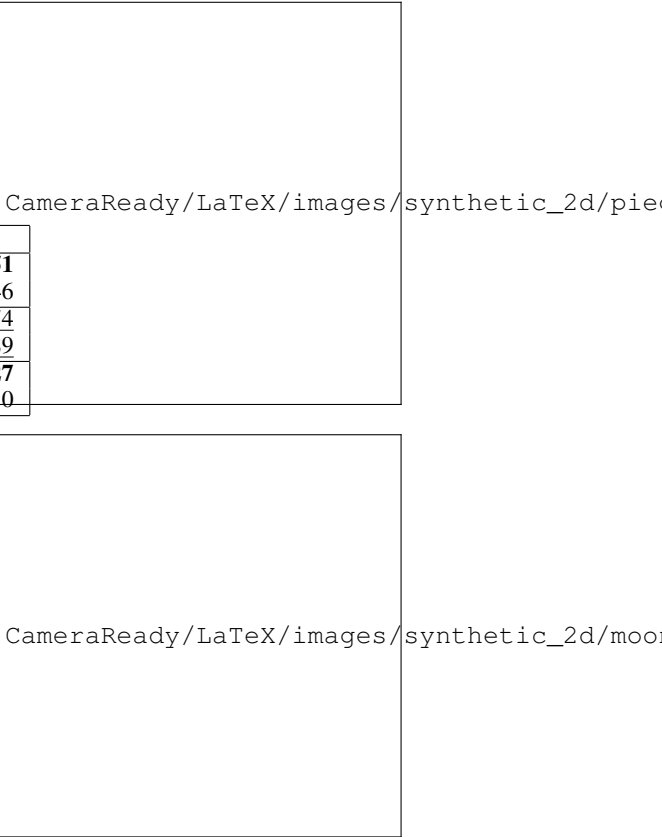


Figure 6: Left to right: reconstructed nodes, reconstructed prior. Top to bottom: PieClam in  $\mathcal{T}$ , PCLam in  $\mathbb{R}_+^{\mathcal{L}}$



Figure 7: Left to right: Target SBM. Fitted BigClam graph with two communities. Error as a function of optimization iteration where error is, left to right, log cut distance, cut distance, l2 distance. After convergence, the log cut distance between the SBM and BigClam is 0.0776.



Figure 8: Left to right: Target SBM. Fitted BigClam graph with four communities. Error as a function of optimization iteration where error is, left to right, log cut distance, cut distance, l2 distance. After convergence, the log cut distance between the SBM and BigClam is 0.0775.

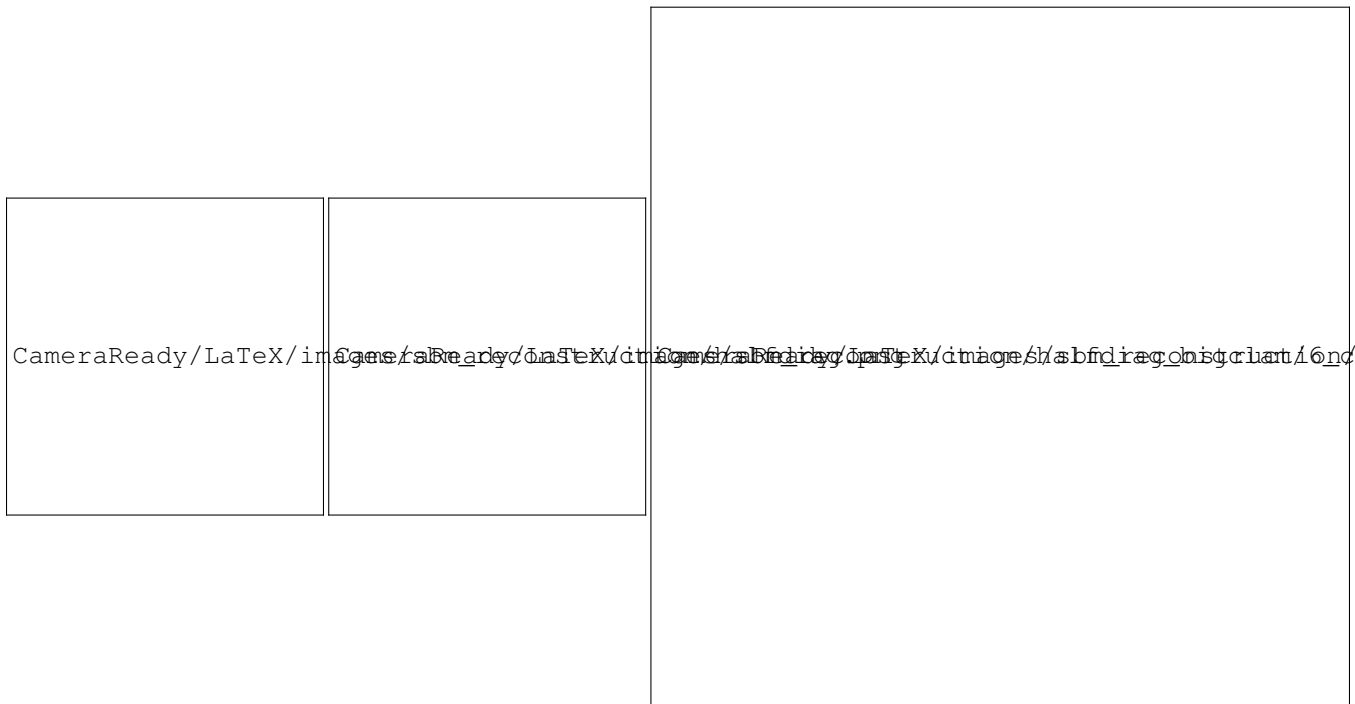


Figure 9: Left to right: Target SBM. Fitted BigClam graph with six communities. Error as a function of optimization iteration where error is, left to right, log cut distance, cut distance, l2 distance. After convergence, the log cut distance between the SBM and BigClam is 0.0776.

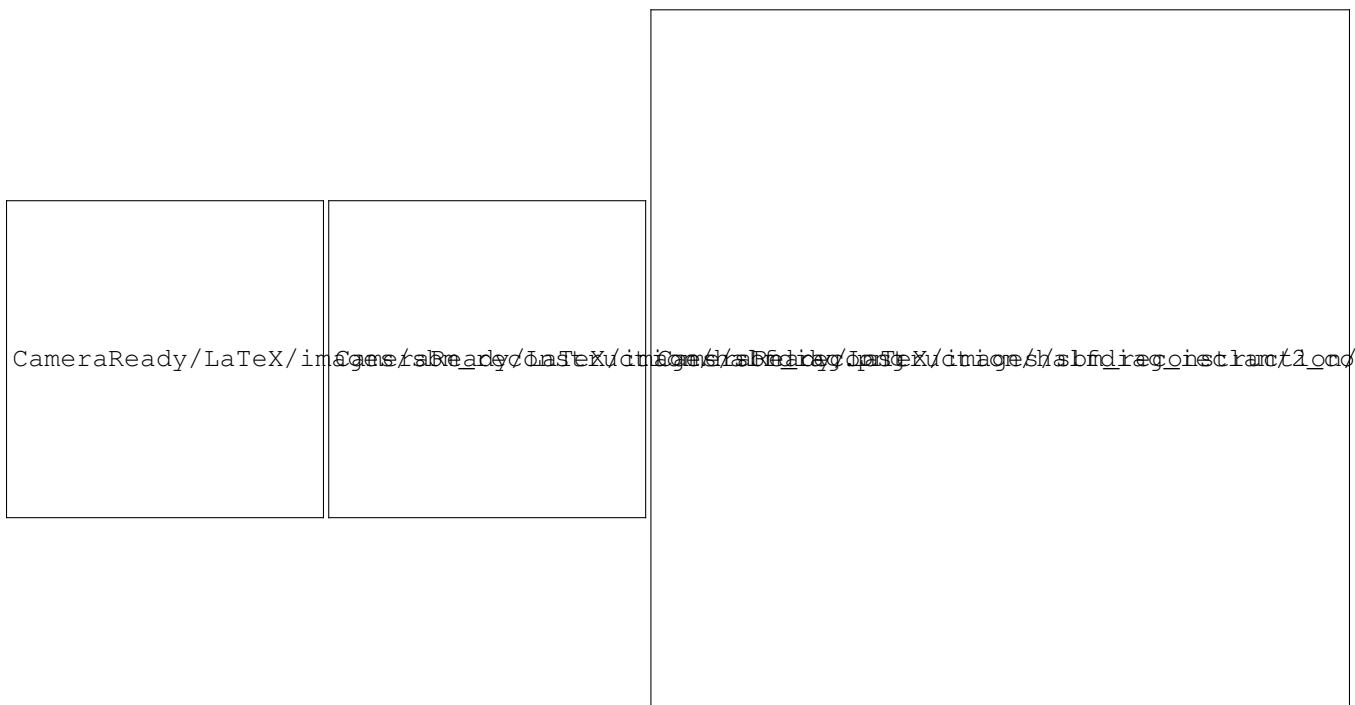


Figure 10: Left to right: Target SBM. Fitted IeClam graph with two communities. Error as a function of optimization iteration where error is, left to right, log cut distance, cut distance, l2 distance. After convergence, the log cut distance between the SBM and IeClam is 0.0662.



Figure 11: Left to right: Target SBM. Fitted IeClam graph with four communities (two inclusive and two exclusive). Error as a function of optimization iteration where error is, left to right, log cut distance, cut distance, l2 distance. After convergence, the log cut distance between the SBM and IeClam is 0.0312.



Figure 12: Left to right: Target SBM. Fitted IeClam graph with six communities (three inclusive and three exclusive). Error as a function of optimization iteration where error is, left to right, log cut distance, cut distance, l2 distance. After convergence, the log cut distance between the SBM and IeClam is 0.0309.





CameraReady/LaTeX/images/synthetic\_2d/moons\_pclam\_reconstructed.png

CameraReady/LaTeX/images/ablation/no\_noise.png

Figure 13: Left to right: reconstructed nodes, reconstructed prior. Top to bottom: With and without noise addition.